



**UNIVERSIDAD
DEL AZUAY**

Facultad de Ciencias de la Administración
Escuela de Ciencias de la Computación
Base de datos II

Tema: Proyecto

Autores: Andrés Merchán, Daniel Maldonado, Marco Rojas, Ariel Serrano

Profesor: PhD. Patricia Ortega

Fecha de elaboración: 28 de Abril de 2026

Sección 2. Introducción y contexto

2.1 Descripción del módulo y su rol dentro del sistema integrado

El Módulo C — Cuentas por Pagar forma parte del Sistema Integrado de Gestión Empresarial (SIGE), diseñado para una pequeña o mediana empresa comercial. Este módulo representa el punto de entrada del ciclo de compra de la organización: gestiona la relación con los proveedores, controla el proceso de adquisición desde la emisión de una orden de compra hasta el registro del pago correspondiente, y garantiza que las obligaciones financieras de la empresa con terceros sean trazables, correctamente registradas y pagadas en los plazos acordados.

Dentro del SIGE, el Módulo C actúa como el origen del flujo de mercadería e insumos. Es el encargado de recibir las solicitudes de reposición generadas por el Módulo B (Control de Inventarios) cuando el stock de un producto cae por debajo del mínimo establecido, convertirlas en órdenes de compra formales, registrar la recepción física de la mercadería y vincularse con el inventario al momento de confirmar dicha recepción. Adicionalmente, gestiona las facturas emitidas por los proveedores y controla los pagos pendientes, parciales y totales.

El flujo principal que implementa este módulo es el siguiente:

Solicitud de compra → Orden de compra → Recepción de mercadería → Factura de proveedor → Registro de pago → Conciliación de saldo

Este ciclo se desarrolla bajo restricciones de integridad referencial y reglas de negocio específicas que se detallan a lo largo del presente informe.

2.2 Entidades compartidas con otros módulos y cómo se gestionan

El SIGE comparte un conjunto de entidades maestras que son utilizadas por los tres módulos. En el caso del Módulo C, las entidades compartidas más relevantes son las siguientes:

Con el Módulo B — Control de Inventarios:

La entidad productos es la principal entidad compartida entre los módulos B y C. El Módulo B es el propietario lógico de esta entidad, pues es quien define el catálogo de productos, sus precios de costo y venta, y controla el stock. El Módulo C la referencia en la tabla `detalle_orden_compra` para indicar qué productos se están solicitando a un proveedor y a qué precio acordado. Al registrar una recepción de mercadería confirmada, el Módulo C genera automáticamente un movimiento de entrada en la tabla `movimientos_inventario` del Módulo B, actualizando el campo `stock_actual` del producto correspondiente. Esta integración se implementa mediante la interfaz IC-B01/IF-03, representada en el diseño como `ServicioInventarioB.notificarMovimiento()`, sin acceso directo a las tablas internas del Módulo B.

De igual forma, cuando el Módulo B detecta que el stock de un producto ha alcanzado su mínimo (`stock_actual <= stock_min`), puede generar una solicitud de compra que el Módulo C procesa para crear una orden de compra.

Con el Módulo A — Facturación y Cuentas por Cobrar:

La relación con el Módulo A es menos directa. Ambos módulos comparten la entidad productos para efectos de cálculo de márgenes y costos de adquisición. No existe una dependencia funcional fuerte entre los dos módulos en el flujo transaccional, pero los datos de costo registrados por el Módulo C (precio acordado en las órdenes de compra) son un insumo de referencia para el Módulo A al momento de establecer precios de venta. Esta integración se formaliza mediante la interfaz IF-01 / ServicioConsultaCostosA, que expone el precio de costo en modo solo lectura sin que el Módulo A acceda directamente a las tablas internas del Módulo C.

Gestión de la integridad referencial entre módulos:

Dado que el SIGE se implementa en una misma base de datos MySQL, las claves foráneas entre tablas de distintos módulos se gestionan directamente a nivel de motor con FOREIGN KEY ... REFERENCES. La tabla productos actúa como entidad central compartida. Para evitar conflictos de propiedad entre módulos, se establece que solo el Módulo B puede ejecutar operaciones de INSERT, UPDATE y DELETE sobre productos; el Módulo C únicamente realiza SELECT sobre esta tabla y notifica el movimiento de entrada al Módulo B mediante la interfaz contractual IC-B01/IF-03 / ServicioInventarioB.notificarMovimiento(), sin acceso directo a las tablas internas del Módulo B.

2.3 Alcance y limitaciones

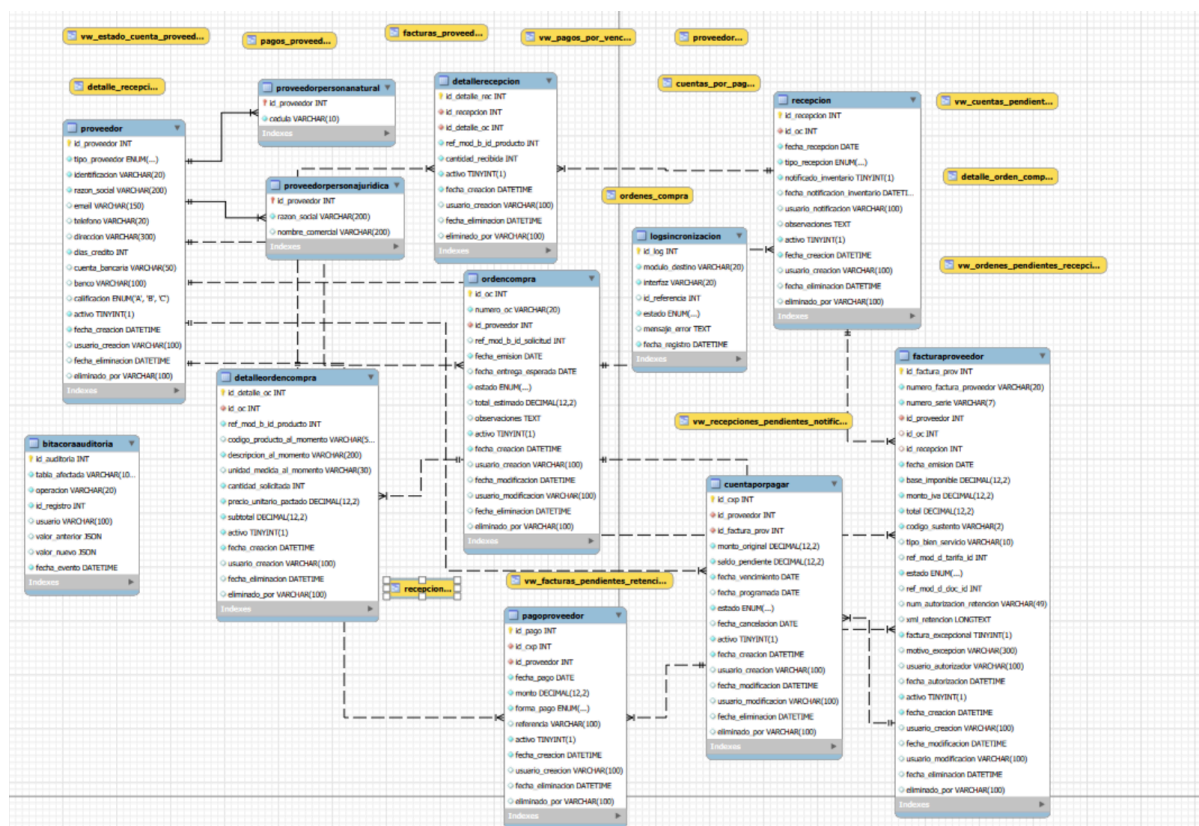
Alcance:

El presente informe cubre el diseño, implementación y optimización de la base de datos correspondiente al Módulo C — Cuentas por Pagar. Incluye el diseño del modelo EER con especialización de la entidad Proveedor, la transformación al esquema relacional, la verificación de la normalización hasta Tercera Forma Normal, la implementación del DDL completo con constraints e índices, los scripts DML de datos de prueba, los objetos de base de datos (vistas, funciones y procedimientos almacenados), la optimización de consultas críticas mediante análisis de planes de ejecución, el diseño de escenarios transaccionales con control de concurrencia, la definición de roles y permisos, y la documentación de la integración con los Módulos A, B y D.

Limitaciones:

- El módulo no implementa autenticación de usuarios a nivel de aplicación; los mecanismos de seguridad se limitan a roles y permisos a nivel de motor de base de datos.
- La integración con el Módulo A se documenta conceptualmente pero no se implementa con transacciones distribuidas, dado que ambos módulos pertenecen a equipos distintos y la integración final se realizará en la etapa de consolidación del SIGE.
- Los datos de prueba son representativos pero no reflejan un volumen de producción real; el análisis de optimización se realiza sobre el conjunto de prueba disponible.
- El sistema asume que la base de datos utilizada es MySQL 8.x con motor InnoDB.

3.1 Diagrama EER del módulo



El diagrama EER del módulo contempla las siguientes decisiones de modelado:

La entidad Proveedor funciona como entidad raíz del modelo. En ella se almacenan los datos comunes de todos los proveedores, tales como identificación, razón social, correo electrónico, teléfono, dirección, días de crédito, cuenta bancaria, banco, calificación, estado activo y datos de auditoría de creación o eliminación lógica.

- **ProveedorPersonaNatural**: representa a los proveedores registrados como personas naturales y contiene como atributo propio la cedula.
- **ProveedorPersonaJuridica**: representa a los proveedores registrados como personas jurídicas y contiene como atributos propios razon social y nombre comercial.

Esta especialización permite separar los datos generales del proveedor de aquellos atributos específicos según el tipo de proveedor. De esta manera, el modelo evita almacenar atributos innecesarios en una sola tabla y mantiene una estructura más clara para consultas, validaciones y reportes.

La especialización se interpreta como disjunta, ya que un proveedor registrado debe pertenecer a un solo tipo: persona natural o persona jurídica. Además, es una especialización total desde el punto de vista funcional, porque todo proveedor válido del sistema debe corresponder a una de esas dos categorías.

Relaciones principales del modelo

El modelo EER representa las siguientes relaciones principales:

- Un Proveedor puede estar asociado a múltiples OrdenCompra, pero cada orden de compra pertenece a un único proveedor.
- Una OrdenCompra puede contener múltiples registros de DetalleOrdenCompra. Cada detalle representa un producto solicitado, la cantidad requerida, el precio unitario pactado y el subtotal correspondiente.
- Una OrdenCompra puede tener una o varias Recepcion, ya que una orden puede recibirse de forma completa o parcial.
- Una Recepcion contiene uno o varios registros de DetalleRecepcion, donde se especifican las cantidades efectivamente recibidas por cada detalle de orden de compra.
- Una FacturaProveedor se relaciona con un Proveedor y puede estar asociada a una OrdenCompra y a una Recepcion. En el modelo también se contempla la posibilidad de facturas excepcionales sin orden de compra asociada, por lo que los campos `id_oc` e `id_recepcion` pueden ser nulos.
- Una FacturaProveedor genera una CuentaPorPagar, que representa la obligación financiera pendiente de pago.
- Una CuentaPorPagar puede tener múltiples registros de PagoProveedor, permitiendo registrar pagos parciales o totales.
- LogSincronizacion permite registrar eventos o errores de integración con otros módulos, especialmente con el Módulo B para notificaciones de inventario y con el Módulo D para procesos tributarios.
- BitacoraAuditoria permite registrar operaciones relevantes sobre datos financieros, almacenando la tabla afectada, operación realizada, usuario, valores anteriores y nuevos, y fecha del evento.

El diagrama también incluye vistas de apoyo para consultas frecuentes del módulo, como cuentas pendientes, pagos por vencer, estado de cuenta por proveedor, facturas pendientes de retención y recepciones pendientes de notificación. Estas vistas no representan entidades principales del dominio, pero sí apoyan los procesos de consulta, reporte e integración del módulo.

3.2 Justificación del modelo

La estructura del modelo responde al flujo real de compras y pagos de una empresa comercial. La separación de entidades permite representar cada etapa del proceso con responsabilidades claras y evita mezclar información operativa, documental y financiera en una sola tabla.

La entidad Proveedor se separa de sus subtipos ProveedorPersonaNatural y ProveedorPersonaJuridica para mantener una especialización ordenada. Los datos comunes permanecen en la tabla principal, mientras que los atributos específicos se almacenan en las tablas derivadas. Esta decisión reduce redundancia y facilita la aplicación de reglas distintas para proveedores naturales y jurídicos.

La separación entre OrdenCompra y DetalleOrdenCompra permite representar correctamente una orden con varios productos. Si los productos se almacenaran directamente en la orden, se generarían grupos repetitivos y se rompería la Primera Forma Normal. Al crear una tabla de detalle, cada producto solicitado queda registrado en una fila independiente con su cantidad y precio pactado.

La separación entre OrdenCompra y Recepcion se justifica porque una orden puede recibirse en varias entregas. Por ejemplo, una compra puede ser emitida por una cantidad total, pero la mercadería puede llegar parcialmente en diferentes fechas. Por eso, la recepción se modela como una entidad independiente relacionada con la orden.

La entidad DetalleRecepcion permite controlar con precisión qué cantidad fue recibida de cada detalle de orden. Esto evita registrar solamente una cantidad general en la cabecera de recepción y permite validar que no se reciba más de lo solicitado.

La entidad FacturaProveedor se separa de CuentaPorPagar porque la factura representa el documento tributario emitido por el proveedor, mientras que la cuenta por pagar representa la obligación financiera que debe ser controlada hasta su cancelación. Esta separación permite manejar estados tributarios en la factura, como pendiente_retencion o retencion_generada, y estados financieros en la cuenta por pagar, como pendiente, pagada_parcial, pagada o vencida.

La entidad PagoProveedor se relaciona con CuentaPorPagar porque los pagos se aplican sobre una obligación financiera. Aunque la cuenta por pagar nace a partir de una factura, el pago no se registra directamente sobre la factura, sino sobre el saldo pendiente de la cuenta. Esta decisión permite registrar pagos parciales y actualizar el saldo de manera controlada.

Finalmente, LogSincronizacion y BitacoraAuditoria se incluyen como tablas de soporte. La primera permite controlar eventos de integración entre módulos y registrar fallos de comunicación. La segunda permite cumplir con la trazabilidad de operaciones financieras relevantes, registrando cambios importantes realizados sobre el módulo.

3.3 Transformación del EER a esquema relacional

A continuación se presenta la transformación del modelo EER al esquema relacional. Las claves primarias se indican como PK y las claves foráneas como FK.

Tablas propias del Módulo C

- Proveedor
(id_proveedor PK, tipo_proveedor, identificacion, razon_social, email, telefono, direccion, dias_credito, cuenta_bancaria, banco, calificacion, activo, fecha_creacion, usuario_creacion, fecha_eliminacion, eliminado_por)
- ProveedorPersonaNatural
(id_proveedor PK, FK → Proveedor.id_proveedor, cedula)
- ProveedorPersonaJuridica
(id_proveedor PK, FK → Proveedor.id_proveedor, razon_social, nombre_comercial)
- OrdenCompra
(id_oc PK, numero_oc, id_proveedor FK → Proveedor.id_proveedor, ref_mod_b_id_solicitud, fecha_emision, fecha_entrega_esperada, estado, total_estimado, observaciones, activo, fecha_creacion, usuario_creacion, fecha_modificacion, usuario_modificacion, fecha_eliminacion, eliminado_por)
- DetalleOrdenCompra
(id_detalle_oc PK, id_oc FK → OrdenCompra.id_oc, ref_mod_b_id_producto, codigo_producto_al_momento, descripcion_al_momento, unidad_medida_al_momento, cantidad_solicitada, precio_unitario_pactado, subtotal, activo, fecha_creacion, usuario_creacion, fecha_eliminacion, eliminado_por)
- Recepcion
(id_recepcion PK, id_oc FK → OrdenCompra.id_oc, fecha_recepcion, tipo_recepcion, notificado_inventario, fecha_notificacion_inventario, usuario_notificacion, observaciones, activo, fecha_creacion, usuario_creacion, fecha_eliminacion, eliminado_por)
- DetalleRecepcion
(id_detalle_rec PK, id_recepcion FK → Recepcion.id_recepcion, id_detalle_oc FK → DetalleOrdenCompra.id_detalle_oc, ref_mod_b_id_producto, cantidad_recibida, activo, fecha_creacion, usuario_creacion, fecha_eliminacion, eliminado_por)
- FacturaProveedor
(id_factura_prov PK, numero_factura_proveedor, numero_serie, id_proveedor FK → Proveedor.id_proveedor, id_oc FK → OrdenCompra.id_oc, id_recepcion FK → Recepcion.id_recepcion, fecha_emision, base_imponible, monto_iva, total, codigo_sustento, tipo_bien_servicio, ref_mod_d_tarifa_id, estado, ref_mod_d_doc_id, num_autorizacion_retencion, xml_retencion, factura_excepcional, motivo_excepcion, usuario_autorizador, fecha_autorizacion, activo, fecha_creacion, usuario_creacion, fecha_modificacion, usuario_modificacion, fecha_eliminacion, eliminado_por)
- CuentaPorPagar
(id_cxp PK, id_proveedor FK → Proveedor.id_proveedor, id_factura_prov FK → FacturaProveedor.id_factura_prov, monto_original, saldo_pendiente, fecha_vencimiento, fecha_programada, estado, fecha_cancelacion, activo, fecha_creacion, usuario_creacion, fecha_modificacion, usuario_modificacion, fecha_eliminacion, eliminado_por)
- PagoProveedor
(id_pago PK, id_cxp FK → CuentaPorPagar.id_cxp, id_proveedor FK → Proveedor.id_proveedor, fecha_pago, monto, forma_pago, referencia, activo, fecha_creacion, usuario_creacion, fecha_eliminacion, eliminado_por)
- LogSincronizacion
(id_log PK, modulo_destino, interfaz, id_referencia, estado, mensaje_error, fecha_registro)

- BitacoraAuditoria
(id_auditoria PK, tabla_afectada, operacion, id_registro, usuario, valor_anterior, valor_nuevo, fecha_evento)

Vistas de apoyo del módulo

El modelo también incluye vistas utilizadas para consultas, reportes e integración:

- vw_cuentas_pendientes
Permite consultar cuentas por pagar activas con saldo pendiente.
- vw_pagos_por_vencer
Permite consultar pagos próximos a vencer dentro del horizonte definido.
- vw_estado_cuenta_proveedor
Permite generar el estado de cuenta de un proveedor, mostrando facturas, pagos y saldos.
- vw_ordenes_pendientes_recepcion
Permite consultar órdenes de compra que aún no han sido recibidas completamente.
- vw_recepciones_pendientes_notificar
Permite identificar recepciones que todavía no han sido notificadas al Módulo B para actualización de inventario.
- vw_facturas_pendientes_retencion
Permite consultar facturas pendientes de envío o actualización tributaria con el Módulo D.

Además, el diseño incluye vistas de compatibilidad con nombres en formato snake_case, tales como proveedores, ordenes_compra, detalle_orden_compra, recepciones, detalle_recepcion, facturas_proveedor, cuentas_por_pagar y pagos_proveedor. Estas vistas facilitan la lectura del modelo y la compatibilidad con convenciones de nombres usadas en otros documentos o contratos del proyecto.

3.4 Verificación de la normalización hasta 3FN

La verificación de normalización se realiza sobre las tablas principales del Módulo C. En general, el modelo cumple con Primera, Segunda y Tercera Forma Normal, ya que los atributos son atómicos, las tablas con claves primarias simples no presentan dependencias parciales y los datos dependientes de otras entidades se encuentran separados en sus propias tablas.

Tabla Proveedor — verificación

Clave primaria: id_proveedor

Dependencias funcionales identificadas:

- id_proveedor → tipo_proveedor, identificacion, razon_social, email, telefono, direccion, dias_credito, cuenta_bancaria, banco, calificacion, activo
- identificacion → id_proveedor, debido a que la identificación es única.

Verificación:

- 1FN: Todos los atributos son atómicos. No existen listas ni grupos repetitivos dentro de la tabla. ✓
- 2FN: La clave primaria es simple, por lo que no existen dependencias parciales. ✓
- 3FN: Los datos específicos de persona natural o jurídica no se almacenan mezclados en la tabla principal, sino en tablas especializadas. Esto evita dependencias transitivas y atributos nulos innecesarios. ✓

Tabla ProveedorPersonaNatural — verificación

Clave primaria: id_proveedor

Dependencias funcionales identificadas:

- id_proveedor → cedula

Verificación:

- 1FN: El atributo cedula es atómico. ✓
- 2FN: La clave primaria es simple, por lo que no existen dependencias parciales. ✓
- 3FN: No existen atributos no clave que dependan de otros atributos no clave. ✓

Tabla ProveedorPersonaJuridica — verificación

Clave primaria: id_proveedor

Dependencias funcionales identificadas:

- id_proveedor → razon_social, nombre_comercial

Verificación:

- 1FN: Los atributos son atómicos. ✓
- 2FN: La clave primaria es simple. ✓
- 3FN: No existen dependencias transitivas entre atributos no clave. ✓

Tabla OrdenCompra — verificación

Clave primaria: id_oc

Dependencias funcionales identificadas:

- id_oc → numero_oc, id_proveedor, ref_mod_b_id_solicitud, fecha_emision, fecha_entrega_esperada, estado, total_estimado, observaciones, activo
- numero_oc → id_oc, debido a que el número de orden es único.

Verificación:

- 1FN: Todos los atributos son atómicos. El atributo estado usa valores discretos como borrador, emitida, recibida_parcial, recibida_total y anulada. ✓
- 2FN: La clave primaria es simple, por lo que no existen dependencias parciales. ✓
- 3FN: Los datos del proveedor no se almacenan en la orden, sino que se referencian mediante id_proveedor. Por ello, no hay dependencia transitiva entre atributos de la orden y datos del proveedor. ✓

Tabla DetalleOrdenCompra — verificación

Clave primaria: id_detalle_oc

Dependencias funcionales identificadas:

- id_detalle_oc → id_oc, ref_mod_b_id_producto, codigo_producto_al_momento, descripcion_al_momento, unidad_medida_al_momento, cantidad_solicitada, precio_unitario_pactado, subtotal

Verificación:

- 1FN: Cada fila representa un único producto solicitado dentro de una orden. Los atributos son atómicos. ✓
- 2FN: La clave primaria es simple, por lo que no existen dependencias parciales. ✓
- 3FN: El precio pactado, la descripción y la unidad de medida se almacenan como una captura del producto al momento de emitir la orden. Esto evita depender de cambios posteriores en el catálogo del Módulo B y mantiene trazabilidad histórica de la compra. ✓

Tabla Recepcion — verificación

Clave primaria: id_recepcion

Dependencias funcionales identificadas:

- id_recepcion → id_oc, fecha_recepcion, tipo_recepcion, notificado_inventario, fecha_notificacion_inventario, usuario_notificacion, observaciones, activo

Verificación:

- 1FN: Los atributos son atómicos. La recepción registra la cabecera del evento de recepción, no una lista de productos. ✓
- 2FN: La clave primaria es simple, por lo que no existen dependencias parciales. ✓
- 3FN: Las cantidades recibidas no se almacenan en la cabecera de recepción, sino en DetalleRecepcion. Esto evita grupos repetitivos y separa correctamente la recepción general de sus productos recibidos. ✓

Tabla DetalleRecepcion — verificación

Clave primaria: id_detalle_rec

Dependencias funcionales identificadas:

- id_detalle_rec → id_recepcion, id_detalle_oc, ref_mod_b_id_producto, cantidad_recibida

Verificación:

- 1FN: Cada fila representa una cantidad recibida para un detalle específico de orden de compra. ✓
- 2FN: La clave primaria es simple. ✓
- 3FN: La tabla referencia al detalle de orden mediante id_detalle_oc, evitando duplicar toda la información de la orden o del producto solicitado. ✓

Tabla FacturaProveedor — verificación

Clave primaria: id_factura_prov

Dependencias funcionales identificadas:

- id_factura_prov → numero_factura_proveedor, numero_serie, id_proveedor, id_oc, id_recepcion, fecha_emision, base_imponible, monto_iva, total, codigo_sustento, tipo_bien_servicio, ref_mod_d_tarifa_id, estado, ref_mod_d_doc_id, num_autorizacion_retencion, xml_retencion, factura_excepcional, motivo_excepcion, usuario_autorizador, fecha_autorizacion

Verificación:

- 1FN: Los atributos son atómicos. Los valores tributarios y de retención se almacenan en campos independientes. ✓
- 2FN: La clave primaria es simple, por lo que no existen dependencias parciales. ✓
- 3FN: Los datos del proveedor, de la orden y de la recepción no se duplican dentro de la factura, sino que se referencian mediante claves foráneas. El estado de la factura se limita al proceso tributario, mientras que el estado financiero se maneja en CuentaPorPagar. ✓

Tabla CuentaPorPagar — verificación

Clave primaria: id_exp

Dependencias funcionales identificadas:

- id_exp → id_proveedor, id_factura_prov, monto_original, saldo_pendiente, fecha_vencimiento, fecha_programada, estado, fecha_cancelacion, activo
- id_factura_prov → id_exp, debido a que una factura válida genera una cuenta por pagar asociada.

Verificación:

- 1FN: Todos los atributos son atómicos. ✓
- 2FN: La clave primaria es simple. ✓
- 3FN: El proveedor se almacena como referencia para facilitar consultas financieras, pero la obligación nace de la factura mediante id_factura_prov. El campo saldo_pendiente puede derivarse del monto original menos los pagos registrados; sin embargo, se almacena como dato controlado para optimizar consultas de saldos, reportes y alertas. Su consistencia se mantiene mediante procedimientos almacenados de registro de pagos y conciliación de saldos, no mediante triggers. ✓

Tabla PagoProveedor — verificación

Clave primaria: id_pago

Dependencias funcionales identificadas:

- id_pago → id_cxp, id_proveedor, fecha_pago, monto, forma_pago, referencia, activo

Verificación:

- 1FN: Cada fila representa un único pago realizado. ✓
- 2FN: La clave primaria es simple. ✓
- 3FN: El pago se asocia a una cuenta por pagar mediante id_cxp, evitando duplicar información completa de la factura. El proveedor se mantiene como referencia directa para facilitar consultas y reportes, sin reemplazar la relación principal con CuentaPorPagar. ✓

Tabla LogSincronizacion — verificación

Clave primaria: id_log

Dependencias funcionales identificadas:

- id_log → modulo_destino, interfaz, id_referencia, estado, mensaje_error, fecha_registro

Verificación:

- 1FN: Todos los atributos son atómicos. ✓
- 2FN: La clave primaria es simple. ✓
- 3FN: La información registrada depende únicamente del evento de sincronización identificado por id_log. ✓

Tabla BitacoraAuditoria — verificación

Clave primaria: id_auditoria

Dependencias funcionales identificadas:

- id_auditoria → tabla_afectada, operacion, id_registro, usuario, valor_anterior, valor_nuevo, fecha_evento

Verificación:

- 1FN: Cada fila representa un evento de auditoría. Aunque valor_anterior y valor_nuevo se almacenan como JSON, cada campo representa un documento estructurado de auditoría y no una lista repetitiva dentro del modelo transaccional principal. ✓
- 2FN: La clave primaria es simple. ✓
- 3FN: Los datos del evento de auditoría dependen directamente de id_auditoria y no de otros atributos no clave. ✓

Sección 4: Implementación SQL

Script DDL completo (CREATE TABLE, constraints, índices, claves foráneas). Script DML con datos de prueba representativos (mínimo 50 registros por tabla principal). Código de objetos de base de datos: triggers, vistas, funciones almacenadas y procedimientos almacenados implementados, con descripción de su propósito y justificación de haberlos elegido.

<https://github.com/angeluda/SIGE/blob/main/ScriptsModuloC>

Sección 5: Optimización de consultas

Mínimo tres consultas relevantes al módulo, con: enunciado en lenguaje natural, sentencia SQL, salida del EXPLAIN/EXPLAIN ANALYZE, análisis del plan de ejecución (tipo de acceso, índices usados, costo estimado), comparación con versión alternativa si aplica, y conclusión sobre por qué la consulta es eficiente o qué se optimizó.

1. Consulta: Consultar los pagos próximos a vencer dentro de un horizonte configurable, mostrando proveedor, factura, saldo pendiente, fecha de vencimiento y días restantes.

SELECT

c.id_cxp,

p.razon_social,

p.identificacion,

```

f.numero_factura_proveedor,

c.saldo_pendiente,

c.fecha_vencimiento,

c.fecha_programada,

fn_dias_vencimiento(c.fecha_vencimiento) AS dias_para_vencer

FROM CuentaPorPagar c

JOIN Proveedor p ON p.id_proveedor = c.id_proveedor

JOIN FacturaProveedor f ON f.id_factura_prov = c.id_factura_prov

WHERE c.activo = 1

AND c.estado IN ('pendiente','pagada_parcial')

AND c.saldo_pendiente > 0

AND c.fecha_vencimiento BETWEEN CURDATE() AND DATE_ADD(CURDATE(),
INTERVAL 7 DAY)

ORDER BY c.fecha_vencimiento ASC;

```

Explain y ExplainAnalyze resultados:

Result Grid Filter Rows: Exports: Wrap Cell Contents:										
	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows
	1	SIMPLE	c	NULL	range	uq_exp_factura,idx_exp_factura,idx_exp_prov...	idx_exp_vencimiento	3	NULL	1
	1	SIMPLE	p	NULL	eq_ref	PRIMARY	PRIMARY	4	modulo_c.c.id_proveedor	1
	1	SIMPLE	f	NULL	eq_ref	PRIMARY	PRIMARY	4	modulo_c.c.id_factura_prov	1
									filtered	Extra
									16.66	Using index condition; Using where
									100.00	NULL
									100.00	NULL

Result Grid Filter Rows: Exports: Wrap Cell Contents:										
EXPLAIN										
-> Nested loop inner join (cost=0.827 rows=0.167) (actual time=0.0447..0.0447 rows=0 loops=1) -> Nested loop inner join (cost=0.768 rows=0.167) (actual time=0.0427..0.0427 rows=0 loops=1) -> Filter: ((c.activo = 1) and (c.estado in ('pendiente','pagada_parcial'))) and (c.saldo_pendiente > 0.00)) (cost=0.71 rows=0.167) (actual time=0.0407..0.0407 rows=0 loops=1) -> Index range scan on c using idx_exp_vencimiento over (2026-05-25' <= fecha_vencimiento <= '2026-06-01'), with index condition: (c.fecha_vencimiento between <cache>{(curdate() and <cache>{(curdate() + interval 7 day))}) (cost=0.71 rows=1) (actual time=0.0395..0.0395 rows=0 loops=1) -> Single-row index lookup on p using PRIMARY (id_proveedor=c.id_proveedor) (cost=0.85 rows=1) (never executed) -> Single-row index lookup on f using PRIMARY (id_factura_prov=c.id_factura_prov) (cost=0.85 rows=1) (never executed)										

Análisis:

El plan de ejecución muestra un Nested loop inner join, donde la tabla principal es CuentaPorPagar, identificada con el alias c. El acceso a esta tabla se realiza mediante un Index range scan usando el índice idx_exp_vencimiento.

El plan indica:

Index range scan on c using idx_exp_vencimiento

Esto significa que MySQL no revisa toda la tabla, sino únicamente el rango de fechas definido por la condición:

```
c.fecha_vencimiento BETWEEN CURDATE()
```

```
AND DATE_ADD(CURDATE(), INTERVAL 7 DAY)
```

El costo estimado de esta parte es bajo:

```
cost=0.71 rows=1
```

y el tiempo real también es bajo:

```
actual time=0.0398..0.0398 rows=0 loops=1
```

En esta ejecución no se encontraron cuentas por pagar dentro del rango de vencimiento consultado, por eso el resultado muestra rows=0. Debido a eso, las búsquedas posteriores en Proveedor y FacturaProveedor aparecen como:

```
never executed
```

Esto no representa un error, sino que simplemente no fue necesario ejecutar los JOIN porque no hubo registros iniciales en CuentaPorPagar que cumplieran las condiciones.

También se observa que el plan aplica filtros adicionales:

```
c.activo = 1
```

```
c.estado IN ('pendiente','pagada_parcial')
```

```
c.saldo_pendiente > 0.00
```

El tipo de acceso es adecuado porque la consulta filtra primero por una columna indexada y reduce el conjunto de datos antes de unir con las demás tablas.

Cambios (versión alternativa si aplica):

La consulta ya está optimizada porque usa directamente fecha_vencimiento en el WHERE, permitiendo el uso del índice idx_cxp_vencimiento.

Una versión menos eficiente sería:

```
WHERE fn_dias_vencimiento(c.fecha_vencimiento) BETWEEN 0 AND 7
```

Esa alternativa no sería recomendable, porque aplicar una función sobre la columna fecha_vencimiento dificultaría el uso directo del índice y obligaría al motor a calcular la función por cada fila antes de filtrar.

La versión actual es mejor porque permite un acceso por rango sobre el índice.

Versión Corregida:

SELECT

c.id_cxp,

p.razon_social,

p.identificacion,

f.numero_factura_proveedor,

c.saldo_pendiente,

c.fecha_vencimiento,

c.fecha_programada,

DATEDIFF(c.fecha_vencimiento, CURDATE()) AS dias_para_vencer

FROM CuentaPorPagar c

JOIN Proveedor p

ON p.id_proveedor = c.id_proveedor

JOIN FacturaProveedor f

ON f.id_factura_prov = c.id_factura_prov

WHERE c.activo = 1

AND c.estado IN ('pendiente','pagada_parcial')

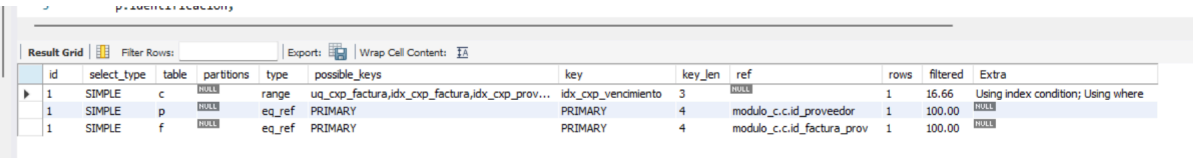
AND c.saldo_pendiente > 0

AND c.fecha_vencimiento BETWEEN CURDATE()

AND DATE_ADD(CURDATE(), INTERVAL 7 DAY)

ORDER BY c.fecha_vencimiento ASC;

Resultados corregidos:



The screenshot shows a database query result grid with the following data:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	c	INDEX	range	uq_cxp_factura,idx_cxp_factura,idx_cxp_prov...	idx_cxp_vencimiento	3	INDEX	1	16.66	Using index condition; Using where
1	SIMPLE	p	INDEX	eq_ref	PRIMARY	PRIMARY	4	modulo_c.c.id_proveedor	1	100.00	INDEX
1	SIMPLE	f	INDEX	eq_ref	PRIMARY	PRIMARY	4	modulo_c.c.id_factura_prov	1	100.00	INDEX

Result Grid		Filter Rows:	Export:	Wrap Cell Contents:
EXPLAIN				
-> Nested loop inner join (cost=0.827 rows=0.167) (actual time=0.0185..0.0185 rows=0 loops=1)				
-> Nested loop inner join (cost=0.768 rows=0.167) (actual time=0.0181..0.0181 rows=0 loops=1)				
-> Filter: ((c.activo = 1) and (c.estado in (pendiente, pagada, parcial)) and (c.saldo_pendiente > 0.00)) (cost=0.71 rows=0.167) (actual time=0.0176..0.0176 rows=0 loops=1)				
-> Index range scan on c using idx_cxp_vencimiento over ('2026-05-25' <= fecha_vencimiento <= '2026-06-01'), with index condition: (c.fecha_vencimiento <cache>((curdate() + interval 7 day))) (cost=0.71 rows=1) (actual time=0.0171..0.0171 rows=0 loops=1)				
-> Single-row index lookup on p using PRIMARY (id_proveedor=c.id_proveedor) (cost=0.85 rows=1) (never executed)				
-> Single-row index lookup on f using PRIMARY (id_factura_prov=c.id_factura_prov) (cost=0.85 rows=1) (never executed)				

Conclusión:

La consulta de pagos próximos a vencer está correctamente optimizada. El plan de ejecución muestra que MySQL utiliza un acceso de tipo range sobre la tabla CuentaPorPagar, usando el índice idx_cxp_vencimiento. Esto confirma que el filtro por fecha_vencimiento se resuelve mediante búsqueda por rango y no mediante un escaneo completo de la tabla.

Aunque el EXPLAIN ANALYZE muestra actual rows = 0, esto no representa un error en la consulta. Simplemente indica que, al momento de la prueba, no existían cuentas por pagar activas, pendientes o parcialmente pagadas cuyo vencimiento estuviera dentro de los próximos siete días. Por esa razón, las búsquedas hacia Proveedor y FacturaProveedor aparecen como never executed, ya que no había registros iniciales sobre los cuales aplicar los JOIN.

En conclusión, la consulta es eficiente porque aprovecha el índice sobre fecha_vencimiento, reduce el conjunto de búsqueda desde el inicio y evita recorrer toda la tabla CuentaPorPagar. La optimización aplicada es adecuada para el reporte de pagos próximos a vencer.

2.Consulta: Consultar las órdenes de compra pendientes de recepción, mostrando proveedor, estado, fecha de emisión, fecha esperada, porcentaje recibido y días transcurridos desde la emisión.

SELECT

o.id_oc,

o.numero_oc,

p.razon_social,

p.identificacion,

o.fecha_emision,

o.fecha_entrega_esperada,

o.estado,

fn_porcentaje_recepcion_oc(o.id_oc) AS porcentaje_recibido,

DATEDIFF(CURDATE(), o.fecha_emision) AS dias_desde_emision,

o.total_estimado

FROM OrdenCompra o

JOIN Proveedor p ON p.id_proveedor = o.id_proveedor

WHERE o.activo = 1

AND o.estado IN ('emitida', 'recibida_parcial')

ORDER BY o.fecha_emision ASC;

Explain y ExplainAnalyze resultados:

The image shows two screenshots from a MySQL database client. The top screenshot displays the 'EXPLAIN' output for a query, showing a nested loop join between the 'OrdenCompra' table (alias 'o') and the 'Proveedor' table (alias 'p'). The bottom screenshot displays the 'EXPLAIN ANALYZE' output, providing detailed execution statistics for the same query.

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	o	NONE	range	idx_oc_proveedor_idx_oc_estado	idx_oc_estado	1	NONE	10	10.00	Using index condition; Using where; Using filesort
1	SIMPLE	p	NONE	eq_ref	PRIMARY	PRIMARY	4	modulo_c.o.id_proveedor	1	100.00	NONE

EXPLAIN	
->	Nested loop inner join (cost=6.26 rows=10) (actual time=0.146..0.164 rows=10 loops=1)
->	Sort: o.fecha_emision (cost=5.01 rows=10) (actual time=0.134..0.136 rows=10 loops=1)
->	Filter: (o.activo = 1) (cost=5.01 rows=10) (actual time=0.0362..0.0468 rows=10 loops=1)
->	Index range scan on o using idx_oc_estado over (estado = 'emitida') OR (estado = 'recibida_parcial'), with index condition: (o.estado in ('emitida','recibida_parcial')) (cost=5.01 rows=10) (actual time=0.0351..0.045 rows=10 loops=1)
->	Single-row index lookup on p using PRIMARY (id_proveedor=o.id_proveedor) (cost=0.35 rows=1) (actual time=0.00248..0.00253 rows=1 loops=10)

Análisis:

El plan de ejecución muestra un Nested loop inner join con un costo estimado total de:

cost=6.26 rows=10

y una ejecución real de:

actual time=0.146..0.164 rows=10 loops=1

La tabla principal es OrdenCompra, con alias o. El acceso a esta tabla se realiza mediante:

Index range scan on o using idx_oc_estado

El índice utilizado es idx_oc_estado, aplicado sobre el campo estado. Esto es coherente con el filtro:

o.estado IN ('emitida', 'recibida_parcial')

El plan indica que MySQL revisa aproximadamente 10 filas:

cost=5.01 rows=10

actual time=0.0351..0.045 rows=10 loops=1

Luego aplica el filtro:

Filter: (o.activo = 1)

Después, el resultado se ordena por fecha de emisión:

Sort: o.fecha_emision

Esto ocurre porque el índice usado ayuda a filtrar por estado, pero no resuelve completamente el ordenamiento por fecha_emision.

La tabla Proveedor, con alias p, se consulta mediante:

Single-row index lookup on p using PRIMARY

Esto significa que por cada orden encontrada se busca exactamente un proveedor mediante la clave primaria id_proveedor. El tiempo real de esta búsqueda es muy bajo:

actual time=0.00248..0.00253 rows=1 loops=10

Por tanto, el acceso a proveedor es eficiente.

Cambios (versión alternativa si aplica):

La consulta ya usa un índice útil (idx_oc_estado), pero puede optimizarse más para reducir el paso de ordenamiento por fecha_emision.

Una mejora posible sería crear un índice compuesto:

```
CREATE INDEX idx_oc_estado_fecha
```

```
ON OrdenCompra (estado, fecha_emision);
```

Una alternativa todavía más ajustada al WHERE sería:

```
CREATE INDEX idx_oc_activo_estado_fecha
```

```
ON OrdenCompra (activo, estado, fecha_emision);
```

Este índice permitiría filtrar por activo, luego por estado, y finalmente facilitar el ordenamiento por fecha_emision.

Versión Corregida:

```
CREATE INDEX idx_oc_activo_estado_fecha
```

```
ON OrdenCompra (activo, estado, fecha_emision);
```

```
EXPLAIN/ EXPLAIN ANALYZE
```

```
SELECT
```

```

o.id_oc,
o.numero_oc,
p.razon_social,
p.identificacion,
o.fecha_emision,
o.fecha_entrega_esperada,
o.estado,
fn_porcentaje_recepcion_oc(o.id_oc) AS porcentaje_recibido,
DATEDIFF(CURDATE(), o.fecha_emision) AS dias_desde_emision,
o.total_estimado
FROM OrdenCompra o
JOIN Proveedor p
    ON p.id_proveedor = o.id_proveedor
WHERE o.activo = 1
    AND o.estado IN ('emitida', 'recibida_parcial')
ORDER BY o.fecha_emision ASC;

```

Resultados corregidos:

Result Grid Filter Rows: Exports: Wrap Cell Contents:										
id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered
1	SIMPLE	o	INDEX	range	idx_oc_proveedor,idx_oc_estado,idx_oc_activ...	idx_oc_estado	1	INDEX	10	100.00
1	SIMPLE	p	INDEX	eq_ref	PRIMARY	PRIMARY	4	modulo_c.o.id_proveedor	1	100.00
Extra										
Using index condition; Using where; Using filesort										

Result Grid Filter Rows: Exports: Wrap Cell Contents:										
EXPLAIN										
-> Nested loop inner join (cost=8.51 rows=10) (actual time=0.277..0.322 rows=10 loops=1)										
-> Sort: o.fecha_emision (cost=5.01 rows=10) (actual time=0.262..0.268 rows=10 loops=1)										
-> Filter: (o.activo = 1) (cost=5.01 rows=10) (actual time=0.0544..0.0922 rows=10 loops=1)										
-> Index range scan on o using idx_oc_estado over (estado = 'emitida') OR (estado = 'recibida_parcial'), with index condition: (o.estado in ('emitida','recibida_parcial')) (cost=5.01 rows=10) (actual time=0.0527..0.0875 rows=10 loops=1)										
-> Single-row index lookup on p using PRIMARY (id_proveedor=o.id_proveedor) (cost=0.26 rows=1) (actual time=0.00472..0.00479 rows=1 loops=10)										

Conclusión:

La consulta de órdenes de compra pendientes de recepción también presenta un plan de ejecución adecuado. MySQL accede a la tabla OrdenCompra mediante un acceso de tipo range, utilizando el índice idx_oc_estado, lo cual permite filtrar directamente las órdenes en estado emitida o recibida_parcial.

El EXPLAIN ANALYZE muestra que la consulta devuelve 10 filas reales y que el acceso a la tabla Proveedor se realiza mediante eq_ref usando la clave primaria, lo cual es eficiente para la relación entre orden de compra y proveedor.

El único punto mejorable es la aparición de Using filesort, causada por el ordenamiento según fecha_emision. Esto no representa un problema importante para el volumen actual de datos, ya que solo se ordenan 10 registros. Sin embargo, si la tabla creciera en producción, podría optimizarse creando un índice compuesto sobre activo, estado y fecha_emision.

En conclusión, la consulta es eficiente para el escenario actual porque usa correctamente el índice idx_oc_estado y evita un escaneo completo de órdenes. La mejora opcional sería agregar un índice compuesto para reducir el costo del ordenamiento por fecha de emisión.

3. Consulta: Generar el reporte de obligaciones pendientes dentro de un rango de fechas, mostrando proveedor, factura, saldo pendiente, fecha de vencimiento, programación de pago, estado y días para vencer.

```
SELECT
    razon_social,
    identificacion,
    numero_factura_proveedor,
    numero_serie,
    monto_original,
    saldo_pendiente,
    fecha_vencimiento,
    fecha_programada,
    estado,
    dias_para_vencer
FROM vw_cuentas_pendientes
WHERE fecha_vencimiento BETWEEN '2025-01-01' AND '2025-12-31'
ORDER BY fecha_vencimiento ASC;
```

Explain y ExplainAnalyze resultados:

Result Grid												
Filter Rows:												
Export: Wrap Cell Content:												
	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	c	NONE	ALL	uq_cxp_factura_idx_cxp_factura_idx_cxp_prov...	NONE	NONE	NONE	55	36.82	Using where; Using filesort
	1	SIMPLE	p	NONE	eq_ref	PRIMARY	PRIMARY	4	modulo_c.c.id_proveedor	1	100.00	NONE
	1	SIMPLE	f	NONE	eq_ref	PRIMARY	PRIMARY	4	modulo_c.c.id_factura_prov	1	100.00	NONE

Result Grid										
Filter Rows:										
Export: Wrap Cell Content:										
EXPLAIN										
-> Nested loop inner join (cost=26.9 rows=55) (actual time=0.0813..0.127 rows=23 loops=1) -> Nested loop inner join (cost=16.3 rows=55) (actual time=0.0731..0.097 rows=23 loops=1) -> Sort: fecha_vencimiento (cost=5.75 rows=55) (actual time=0.0638..0.0655 rows=23 loops=1) -> Filter: ((c.activo = 1) and (c.fecha_vencimiento between '2025-01-01' and '2025-12-31') and (c.estado <> 'pagada')) (cost=5.75 rows=55) (actual time=0.0411..0.051 rows=23 loops=1) -> Table scan on c (cost=5.75 rows=55) (actual time=0.0279..0.0331 rows=55 loops=1) -> Single-row index lookup on p using PRIMARY (id_proveedor=c.id_proveedor) (cost=0.255 rows=1) (actual time=0.00123..0.00124 rows=1 loops=23) -> Single-row index lookup on f using PRIMARY (id_factura_prov=c.id_factura_prov) (cost=0.255 rows=1) (actual time=0.00121..0.00122 rows=1 loops=23)										

Análisis:

El plan de ejecución muestra un Nested loop inner join con costo estimado total de:

cost=26.9 rows=55

y ejecución real de:

actual time=0.0813..0.127 rows=23 loops=1

La consulta parte de la vista vw_cuentas_pendientes, pero internamente el motor trabaja sobre la tabla CuentaPorPagar, identificada como c.

En esta consulta, el acceso a CuentaPorPagar se realiza mediante:

Table scan on c

Esto significa que MySQL realiza un escaneo completo de la tabla CuentaPorPagar. El plan estima revisar 55 filas:

cost=5.75 rows=55

actual time=0.0279..0.0331 rows=55 loops=1

Luego aplica el filtro:

Filter: ((c.activo = 1)

and (c.fecha_vencimiento between '2025-01-01' and '2025-12-31')

and (c.estado <> 'pagada'))

Después del filtro, el resultado baja a 23 filas:

actual time=0.0411..0.051 rows=23 loops=1

También aparece un paso de ordenamiento:

Sort: fecha_vencimiento

Esto ocurre por el ORDER BY fecha_vencimiento ASC.

Las tablas Proveedor y FacturaProveedor se acceden mediante búsquedas por clave primaria:

Single-row index lookup on p using PRIMARY

Single-row index lookup on f using PRIMARY

Estas búsquedas son eficientes, porque por cada cuenta por pagar encontrada se recupera un único proveedor y una única factura relacionada.

El punto menos eficiente de esta consulta es el acceso inicial a CuentaPorPagar, porque usa Table scan en lugar de un índice por fecha de vencimiento, estado o activo.

Cambios (versión alternativa si aplica):

La consulta funciona correctamente, pero puede optimizarse evitando depender de la vista para el filtrado y consultando directamente las tablas base. Una versión alternativa sería:

SELECT

p.razon_social,

p.identificacion,

f.numero_factura_proveedor,

f.numero_serie,

c.monto_original,

c.saldo_pendiente,

c.fecha_vencimiento,

c.fecha_programada,

c.estado,

DATEDIFF(c.fecha_vencimiento, CURDATE()) AS dias_para_vencer

FROM CuentaPorPagar c

JOIN Proveedor p

ON p.id_proveedor = c.id_proveedor

JOIN FacturaProveedor f

ON f.id_factura_prov = c.id_factura_prov

```
WHERE c.activo = 1
```

```
AND c.estado <> 'pagada'
```

```
AND c.fecha_vencimiento BETWEEN '2025-01-01' AND '2025-12-31'
```

```
ORDER BY c.fecha_vencimiento ASC;
```

Además, para mejorar el rendimiento en reportes de obligaciones pendientes, se recomienda un índice compuesto:

```
CREATE INDEX idx_cxp_reporte_obligaciones
```

```
ON CuentaPorPagar (activo, estado, fecha_vencimiento);
```

También podría considerarse:

```
CREATE INDEX idx_cxp_activo_vencimiento_estado
```

```
ON CuentaPorPagar (activo, fecha_vencimiento, estado);
```

Esta segunda opción puede ser útil porque el rango principal de la consulta está sobre fecha_vencimiento.

Versión Corregida:

```
CREATE INDEX idx_cxp_reporte_obligaciones
```

```
ON CuentaPorPagar (activo, fecha_vencimiento, estado);
```

```
EXPLAIN/ EXPLAIN ANALYZE
```

```
SELECT
```

```
    p.razon_social,
```

```
    p.identificacion,
```

```
    f.numero_factura_proveedor,
```

```
    f.numero_serie,
```

```
    c.monto_original,
```

```
    c.saldo_pendiente,
```

```
    c.fecha_vencimiento,
```

```
    c.fecha_programada,
```



```

c.estado,

DATEDIFF(c.fecha_vencimiento, CURDATE()) AS dias_para_vencer

FROM CuentaPorPagar c

JOIN Proveedor p

ON p.id_proveedor = c.id_proveedor

JOIN FacturaProveedor f

ON f.id_factura_prov = c.id_factura_prov

WHERE c.activo = 1

AND c.estado <> 'pagada'

AND c.fecha_vencimiento BETWEEN '2025-01-01' AND '2025-12-31'

ORDER BY c.fecha_vencimiento ASC;

```

Resultados corregidos:

Result Grid										
Filter Rows:										
Export: Wrap Cell Content: I A										
	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows
	1	SIMPLE	c	NULL	ALL	uq_cxp_factura,idx_cxp_factura,idx_cxp_prov...	NULL	NULL	NULL	55
	1	SIMPLE	p	NULL	eq_ref	PRIMARY	PRIMARY	4	modulo_c.c.id_proveedor	1
	1	SIMPLE	f	NULL	eq_ref	PRIMARY	PRIMARY	4	modulo_c.c.id_factura_prov	1
										filtered
										36.82
										100.00
										100.00
										Extra
										Using where; Using filesort
										NULL
										NULL

EXPLAIN										
-> Nested loop inner join (cost=26.9 rows=55) (actual time=0.0991..0.162 rows=23 loops=1) -> Nested loop inner join (cost=16.3 rows=55) (actual time=0.089..0.119 rows=23 loops=1) -> Sort: c.fecha_vencimiento (cost=5.75 rows=55) (actual time=0.0775..0.0799 rows=23 loops=1) -> Filter: ((c.activo = 1) and (c.estado <> 'pagada') and (c.fecha_vencimiento between '2025-01-01' and '2025-12-31')) (cost=5.75 rows=55) (actual time=0.0397..0.0504 rows=23 loops=1) -> Table scan on c (cost=5.75 rows=55) (actual time=0.0297..0.0353 rows=55 loops=1) -> Single-row index lookup on p using PRIMARY (id_proveedor=c.id_proveedor) (cost=0.255 rows=1) (actual time=0.0016..0.00161 rows=1 loops=23) -> Single-row index lookup on f using PRIMARY (id_factura_prov=c.id_factura_prov) (cost=0.255 rows=1) (actual time=0.00133..0.00136 rows=1 loops=23)										

Conclusión:

La consulta cumple correctamente su función, ya que genera el reporte de obligaciones pendientes dentro del rango de fechas solicitado y relaciona cada cuenta por pagar con su proveedor y factura correspondiente.

Sin embargo, a diferencia de las consultas anteriores, esta no queda completamente optimizada a nivel de acceso inicial porque el plan muestra un Table scan sobre CuentaPorPagar. Para el conjunto de prueba esto no representa un problema grave, ya que solo se revisan 55 filas y el tiempo real de ejecución es bajo. No obstante, en un escenario con mayor cantidad de registros, este comportamiento podría afectar el rendimiento.

La consulta es aceptable para el volumen actual, pero la optimización recomendada sería crear un índice compuesto sobre activo, fecha_vencimiento y estado, con el fin de reducir el escaneo completo y mejorar el filtrado de obligaciones pendientes por fecha de vencimiento.

Sección 6: Gestión de transacciones y concurrencia

Al menos dos escenarios de transacciones que representen operaciones críticas del módulo (p. ej., emisión de factura que descuenta inventario). Para cada escenario: código SQL con BEGIN/COMMIT/ROLLBACK, nivel de aislamiento elegido y justificación(Comitted, Uncommitted, Repeatibleread, serializable), análisis de posibles anomalías que se previenen, prueba documentada del comportamiento ante fallo.

Escenario 1: Ingreso de factura de proveedor y generación automática de cuenta por pagar

Descripción del escenario

Este escenario representa el proceso crítico en el que el Módulo C registra una factura emitida por un proveedor y genera automáticamente la cuenta por pagar correspondiente.

La operación es crítica porque afecta varias tablas relacionadas:

- FacturaProveedor
- CuentaPorPagar
- BitacoraAuditoria
- OrdenCompra, como referencia de control
- Recepcion, como validación previa

Además, el procedimiento valida que la factura no supere el monto pendiente de facturación de la orden de compra. Por eso, se utiliza bloqueo con FOR UPDATE sobre la orden de compra, evitando que dos usuarios registren facturas concurrentes sobre la misma orden y superen el total permitido.

Código SQL de la transacción

El siguiente bloque resume el núcleo transaccional usado en sp_ingresar_factura_proveedor:

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

```
START TRANSACTION;
```

```
-- Validar y bloquear la orden de compra asociada
```

```
SELECT COALESCE(total_estimado, 0)
```

```
INTO v_total_estimado
```

```
FROM OrdenCompra
```

```
WHERE id_oc = p_id_oc
```

```
FOR UPDATE;
```

```
-- Calcular el total ya facturado anteriormente para la misma orden
```

```
SELECT COALESCE(SUM(total), 0)
```

```
INTO v_total_facturado_prev
```

```
FROM FacturaProveedor
```

```
WHERE id_oc = p_id_oc
```

```
AND activo = 1
```

```
AND estado <> 'anulada';
```

```
-- Validar que la nueva factura no exceda el monto pendiente
```

```
IF v_total_facturado_prev + v_total > v_total_estimado THEN
```

```
    ROLLBACK;
```

```
    SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'El monto de la factura supera el valor pendiente de  
facturación de la orden de compra.';
```

```
END IF;
```

```
-- Registrar la factura del proveedor
```

```
INSERT INTO FacturaProveedor (
```

```
    numero_factura_proveedor,
```

```
    numero_serie,
```

```
    id_proveedor,
```

```
    id_oc,
```

```
    id_recepcion,
```

```
    fecha_emision,
```

```
base_imponible,  
monto_iva,  
total,  
codigo_sustento,  
tipo_bien_servicio,  
ref_mod_d_tarifa_id,  
estado,  
factura_excepcional,  
motivo_excepcion,  
usuario_autorizador,  
fecha_autorizacion,  
activo,  
fecha_creacion,  
usuario_creacion  
)
```

```
VALUES (
```

```
    p_numero_factura,  
    p_numero_serie,  
    p_id_proveedor,  
    p_id_oc,  
    p_id_recepcion,  
    CURDATE(),  
    p_base_imponible,  
    p_monto_iva,  
    v_total,  
    p_codigo_sustento,  
    p_tipo_bien_servicio,
```

```
p_ref_mod_d_tarifa_id,  
'pendiente_retencion',  
p_factura_excepcional,  
p_motivo_excepcion,  
CASE WHEN p_factura_excepcional = 1 THEN p_usuario_autorizador ELSE NULL  
END,  
CASE WHEN p_factura_excepcional = 1 THEN NOW() ELSE NULL END,  
1,  
NOW(),  
p_usuario  
);
```

```
SET v_id_factura = LAST_INSERT_ID();
```

```
-- Registrar auditoría de la factura
```

```
INSERT INTO BitacoraAuditoria (
```

```
    tabla_afectada,
```

```
    operacion,
```

```
    id_registro,
```

```
    usuario,
```

```
    valor_nuevo,
```

```
    fecha_evento
```

```
)
```

```
VALUES (
```

```
    'FacturaProveedor',
```

```
    'INSERT',
```

```
    v_id_factura,
```

```

p_usuario,
JSON_OBJECT(
    'numero_factura', p_numero_factura,
    'total', v_total,
    'excepcional', p_factura_excepcional,
    'estado', 'pendiente_retencion'
),
NOW()
);

-- Generar automáticamente la cuenta por pagar asociada
CALL sp_generar_cuenta_por_pagar(v_id_factura);

```

```
COMMIT;
```

En caso de error, el procedimiento usa un manejador de excepción:

```

DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
    ROLLBACK;
    RESIGNAL;
END;

```

Nivel de aislamiento elegido

Nivel elegido: READ COMMITTED.

Justificación

Se elige READ COMMITTED porque evita lecturas sucias y permite trabajar con datos confirmados por otras transacciones. Para este caso no es necesario usar SERIALIZABLE, ya que sería más restrictivo y bloquearía más de lo necesario.

La consistencia crítica se controla mediante:

```
FOR UPDATE
```

sobre la fila de OrdenCompra.

Esto permite que, si dos usuarios intentan facturar al mismo tiempo sobre la misma orden, una transacción bloquee la orden mientras calcula el total ya facturado y registra la nueva factura. La segunda transacción deberá esperar y, cuando continúe, verá los datos ya actualizados.

Anomalías que se previenen

Lectura sucia

Se evita porque READ COMMITTED no permite leer datos no confirmados por otras transacciones.

Actualización perdida

Se evita porque la orden de compra se bloquea con FOR UPDATE. Así, dos transacciones no pueden calcular al mismo tiempo el mismo monto pendiente como si ambas fueran la primera.

Facturación excedida

Se evita porque el procedimiento calcula el total ya facturado y lo compara contra el total estimado de la orden antes de insertar la nueva factura.

Inconsistencia entre factura y cuenta por pagar

Se evita porque la factura y la cuenta por pagar se registran dentro de la misma transacción. Si falla la generación de la cuenta por pagar, también se revierte la factura.

Prueba documentada del comportamiento ante fallo

Caso de prueba

Intentar registrar una factura cuyo total supera el valor pendiente de facturación de la orden de compra.

CALL sp_ingresar_factura_proveedor(

'000999999',

'001-001',

'01',

21,

1,

1,

```

999999.00,
149999.85,
'bien',
1,
0,
NULL,
NULL,
'contador_prueba'
);

```

Resultado esperado

El procedimiento debe cancelar la operación y mostrar un error similar a:

El monto de la factura supera el valor pendiente de facturación de la orden de compra.

122	02:26:41	CALL sp_ingresar_factura_proveedor('000999999', '001-001', '01', 21, 1, 1, 999999.00, 149999.85, 'bien', 1, 0, NULL, NULL, 'contador_prueba')	Error Code: 1644. El monto de la factura supera el valor pendiente de facturación de la orden de compra.	0.000 sec
-----	----------	---	--	-----------

```

02:26:41      CALL sp_ingresar_factura_proveedor( '000999999', '001-001', '01', 21,
1, 1, 999999.00, 149999.85, 'bien', 1, 0, NULL, NULL, 'contador_prueba' )
Error Code: 1644. El monto de la factura supera el valor pendiente de facturación de
la orden de compra. 0.000 sec

```

Verificación posterior

```
SELECT *
```

```
FROM FacturaProveedor
```

```
WHERE numero_factura_proveedor = '000999999';
```

```
SELECT *
```

```
FROM CuentaPorPagar
```

```
WHERE id_factura_prov IN (
```

```
    SELECT id_factura_prov
```

```
    FROM FacturaProveedor
```



```
WHERE numero_factura_proveedor = '000999999'
```

```
);
```

Resultado esperado de la verificación

No debe existir ninguna factura registrada con ese número y tampoco debe generarse una cuenta por pagar asociada. Esto demuestra que el ROLLBACK funciona correctamente y mantiene la consistencia del módulo.

```
124 02:27:49 SELECT * FROM CuentaPorPagar WHERE id_factura_prov IN ( SELECT id_factura_prov FROM FacturaProveedor ... 0 row(s) returned 0.000 sec / 0.000 sec
```

```
02:27:49 SELECT * FROM CuentaPorPagar WHERE id_factura_prov IN ( SELECT id_factura_prov FROM FacturaProveedor WHERE numero_factura_proveedor = '000999999' ) LIMIT 0, 1000 0 row(s) returned 0.000 sec / 0.000 sec
```

Escenario 2: Registro de pago y actualización del saldo pendiente

Descripción del escenario

Este escenario representa el proceso en el que el Tesorero registra un pago a proveedor. La operación es crítica porque modifica directamente el saldo financiero de una cuenta por pagar.

La transacción afecta principalmente las siguientes tablas:

- CuentaPorPagar
- PagoProveedor
- BitacoraAuditoria

El procedimiento debe garantizar que dos pagos concurrentes no actualicen incorrectamente el mismo saldo pendiente. Para esto, el procedimiento `sp_registrar_pago` bloquea la cuenta por pagar con `FOR UPDATE` antes de calcular el nuevo saldo.

Código SQL de la transacción

El siguiente bloque resume el núcleo transaccional usado en `sp_registrar_pago`:

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

```
START TRANSACTION;
```

```
-- Bloquear la cuenta por pagar antes de registrar el pago
```

```
SELECT saldo_pendiente, id_proveedor
```

```
INTO v_saldo, v_id_prov
```

```
FROM CuentaPorPagar
```

```
WHERE id_cxp = p_id_cxp
```

```
AND activo = 1
```

```
FOR UPDATE;
```

```
-- Validar que la cuenta exista
```

```
IF v_saldo IS NULL THEN
```

```
    ROLLBACK;
```

```
    SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'La cuenta por pagar no existe o está inactiva.';
```

```
END IF;
```

```
-- Validar que el pago no supere el saldo pendiente
```

```
IF p_monto > v_saldo THEN
```

```
    ROLLBACK;
```

```
    SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'El monto a pagar supera el saldo pendiente de la cuenta.';
```

```
END IF;
```

```
-- Registrar el pago
```

```
INSERT INTO PagoProveedor (
```

```
    id_cxp,
```

```
    id_proveedor,
```

```
    fecha_pago,
```

```
    monto,
```

```
    forma_pago,
```

```
    referencia,
```

```
    activo,  
    fecha_creacion,  
    usuario_creacion  
)
```

```
VALUES (  
    p_id_exp,  
    v_id_prov,  
    CURDATE(),  
    p_monto,  
    p_forma_pago,  
    p_referencia,  
    1,  
    NOW(),  
    p_usuario  
);
```

```
SET v_id_pago = LAST_INSERT_ID();  
SET v_nuevo_saldo = v_saldo - p_monto;
```

```
-- Actualizar saldo y estado de la cuenta por pagar
```

```
UPDATE CuentaPorPagar
```

```
SET saldo_pendiente = v_nuevo_saldo,
```

```
    estado = CASE
```

```
        WHEN v_nuevo_saldo <= 0 THEN 'pagada'
```

```
        ELSE 'pagada_parcial'
```

```
    END,
```

```
    fecha_cancelacion = CASE
```

```
        WHEN v_nuevo_saldo <= 0 THEN CURDATE()
        ELSE fecha_cancelacion
    END,
    fecha_modificacion = NOW(),
    usuario_modificacion = p_usuario
WHERE id_cxp = p_id_cxp;
```

-- Registrar auditoría del pago

```
INSERT INTO BitacoraAuditoria (
    tabla_afectada,
    operacion,
    id_registro,
    usuario,
    valor_anterior,
    valor_nuevo,
    fecha_evento
)
VALUES (
    'PagoProveedor',
    'INSERT',
    v_id_pago,
    p_usuario,
    JSON_OBJECT('saldo_anterior', v_saldo),
    JSON_OBJECT(
        'monto_pagado', p_monto,
        'saldo_nuevo', v_nuevo_saldo,
        'forma_pago', p_forma_pago
```

```
),  
NOW()  
);
```

```
COMMIT;
```

En caso de error, el procedimiento usa:

```
DECLARE EXIT HANDLER FOR SQLEXCEPTION  
  
BEGIN  
  
    ROLLBACK;  
  
    RESIGNAL;  
  
END;
```

Nivel de aislamiento elegido

Nivel elegido: READ COMMITTED.

Justificación

Se utiliza READ COMMITTED porque evita lecturas sucias y permite trabajar únicamente con saldos confirmados. Para proteger el saldo pendiente de la cuenta, el procedimiento usa:

```
FOR UPDATE
```

sobre la fila de CuentaPorPagar.

Esto es suficiente para evitar que dos pagos concurrentes lean el mismo saldo y lo actualicen de manera incorrecta. No se elige READ UNCOMMITTED porque permitiría leer saldos no confirmados. Tampoco se elige SERIALIZABLE porque bloquearía más registros de los necesarios y reduciría el rendimiento.

Anomalías que se previenen

Lectura sucia

Se evita porque la transacción trabaja bajo READ COMMITTED, por lo que no puede leer cambios no confirmados de otra transacción.

Actualización perdida

Se evita porque la fila de CuentaPorPagar se bloquea con FOR UPDATE. Si dos usuarios intentan pagar la misma cuenta al mismo tiempo, uno debe esperar a que el otro termine.

Sobrepago

Se evita porque el procedimiento compara el monto del pago contra el saldo pendiente antes de insertar el pago.

Inconsistencia entre pago y saldo

Se evita porque el registro del pago y la actualización del saldo se ejecutan dentro de la misma transacción. Si falla cualquier paso, todo se revierte.

Prueba documentada del comportamiento ante fallo

Caso de prueba

Intentar registrar un pago superior al saldo pendiente de una cuenta por pagar.

```
CALL sp_registrar_pago(  
    1,  
    999999.00,  
    'transferencia',  
    'TRX-ERROR-001',  
    'tesorero_prueba'  
);
```

Resultado esperado

El procedimiento debe cancelar la operación y mostrar un error similar a:

El monto a pagar supera el saldo pendiente de la cuenta.

125	02:28:59	CALL sp_registrar_pago(1, 999999.00, 'transferencia', 'TRX-ERROR-001', 'tesorero_prueba')	Error Code: 1644. El monto a pagar supera el saldo pendiente de la cuenta.	0.000 sec
-----	----------	---	--	-----------

```
02:28:59      CALL sp_registrar_pago( 1, 999999.00, 'transferencia',  
'TRX-ERROR-001', 'tesorero_prueba' )      Error Code: 1644. El monto a pagar supera el  
saldo pendiente de la cuenta. 0.000 sec
```

Verificación posterior

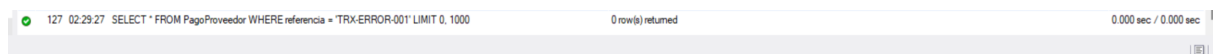
```
SELECT id_cxp, saldo_pendiente, estado  
  
FROM CuentaPorPagar  
  
WHERE id_cxp = 1;
```

```
SELECT *  
  
FROM PagoProveedor  
  
WHERE referencia = 'TRX-ERROR-001';
```

Resultado esperado de la verificación

El saldo de la cuenta por pagar debe mantenerse igual que antes del intento fallido. Además, no debe existir ningún registro en PagoProveedor con la referencia TRX-ERROR-001.

Esto demuestra que el ROLLBACK evita que se registre un pago inválido y mantiene consistente el saldo de la cuenta por pagar.



127 02:29:27 SELECT * FROM PagoProveedor WHERE referencia = 'TRX-ERROR-001' LIMIT 0, 1000 0 row(s) returned 0.000 sec / 0.000 sec

```
02:29:27      SELECT * FROM PagoProveedor WHERE referencia = 'TRX-ERROR-001'  
LIMIT 0, 1000      0 row(s) returned      0.000 sec / 0.000 sec
```

Sección 7: Administración y seguridad

Definición de al menos dos roles con permisos diferenciados para el módulo (GRANT/REVOKE). Estrategia de respaldo: instrucciones de backup y restauración para el módulo. Al menos una consideración de seguridad adicional: auditoría, enmascaramiento de datos sensibles, o prevención de inyección SQL en procedimientos.

7.1 Definición de roles y permisos

Para aplicar control de acceso se definen tres roles diferenciados:

- rol_jefe_compras: gestiona proveedores, órdenes de compra, recepciones y facturas de proveedor.
- rol_tesorero: gestiona pagos, programación de pagos, estados de cuenta y reportes financieros.
- rol_admin_modulo_c: tiene acceso completo a las funciones del módulo, incluyendo autorizaciones excepcionales y tareas administrativas.

Creación de roles

```
CREATE ROLE IF NOT EXISTS 'rol_jefe_compras';  
  
CREATE ROLE IF NOT EXISTS 'rol_tesorero';  
  
CREATE ROLE IF NOT EXISTS 'rol_admin_modulo_c';
```

Permisos para Jefe de Compras

El Jefe de Compras puede ejecutar procedimientos relacionados con el ciclo operativo del módulo: proveedores, órdenes de compra, recepciones y facturas.

```
GRANT EXECUTE ON PROCEDURE modulo_c.sp_registrar_proveedor  
TO 'rol_jefe_compras';
```

```
GRANT EXECUTE ON PROCEDURE modulo_c.sp_actualizar_proveedor  
TO 'rol_jefe_compras';
```

```
GRANT EXECUTE ON PROCEDURE modulo_c.sp_eliminar_proveedor_logico  
TO 'rol_jefe_compras';
```

```
GRANT EXECUTE ON PROCEDURE modulo_c.sp_crear_orden_compra  
TO 'rol_jefe_compras';
```

```
GRANT EXECUTE ON PROCEDURE modulo_c.sp_registrar_recepcion  
TO 'rol_jefe_compras';
```

```
GRANT EXECUTE ON PROCEDURE modulo_c.sp_ingresar_factura_proveedor  
TO 'rol_jefe_compras';
```

```
GRANT EXECUTE ON PROCEDURE  
modulo_c.sp_reporte_ordenes_pendientes_recepcion  
TO 'rol_jefe_compras';
```

También se le permite consultar vistas relacionadas con órdenes, recepciones y facturas pendientes de integración:

```
GRANT SELECT ON modulo_c.vw_ordenes_pendientes_recepcion  
TO 'rol_jefe_compras';
```



```
GRANT SELECT ON modulo_c.vw_recepciones_pendientes_notificar  
TO 'rol_jefe_compras';
```

```
GRANT SELECT ON modulo_c.vw_facturas_pendientes_retencion  
TO 'rol_jefe_compras';
```

Para evitar modificaciones directas sobre tablas, se revocan permisos de escritura directa:

```
REVOKE INSERT, UPDATE, DELETE ON modulo_c.Proveedor  
FROM 'rol_jefe_compras';
```

```
REVOKE INSERT, UPDATE, DELETE ON modulo_c.OrdenCompra  
FROM 'rol_jefe_compras';
```

```
REVOKE INSERT, UPDATE, DELETE ON modulo_c.Recepcion  
FROM 'rol_jefe_compras';
```

```
REVOKE INSERT, UPDATE, DELETE ON modulo_c.FacturaProveedor  
FROM 'rol_jefe_compras';
```

Permisos para Tesorero

El Tesorero puede ejecutar procedimientos relacionados con el ciclo financiero: cuentas por pagar, pagos, saldos, alertas y reportes.

```
GRANT EXECUTE ON PROCEDURE modulo_c.sp_registrar_pago  
TO 'rol_tesorero';
```

```
GRANT EXECUTE ON PROCEDURE modulo_c.sp_programar_pago  
TO 'rol_tesorero';
```

```
GRANT EXECUTE ON PROCEDURE modulo_c.sp_estado_cuenta_proveedor  
TO 'rol_tesorero';
```

```
GRANT EXECUTE ON PROCEDURE modulo_c.sp_reporte_obligaciones_pendientes  
TO 'rol_tesorero';
```

```
GRANT EXECUTE ON PROCEDURE modulo_c.sp_alertar_pagos_por_vencer  
TO 'rol_tesorero';
```

```
GRANT EXECUTE ON PROCEDURE modulo_c.sp_conciliar_saldos_proveedor  
TO 'rol_tesorero';
```

```
GRANT EXECUTE ON PROCEDURE modulo_c.sp_actualizar_cuentas_vencidas  
TO 'rol_tesorero';
```

También puede consultar vistas financieras:

```
GRANT SELECT ON modulo_c.vw_cuentas_pendientes  
TO 'rol_tesorero';
```

```
GRANT SELECT ON modulo_c.vw_pagos_por_vencer  
TO 'rol_tesorero';
```

```
GRANT SELECT ON modulo_c.vw_estado_cuenta_proveedor  
TO 'rol_tesorero';
```

```
GRANT SELECT ON modulo_c.cuentas_por_pagar  
TO 'rol_tesorero';
```

```
GRANT SELECT ON modulo_c.pagos_proveedor
```

```
TO 'rol_tesorero';
```

Se revoca acceso directo de modificación a tablas sensibles para obligar a que los pagos y saldos se gestionen mediante procedimientos almacenados:

```
REVOKE INSERT, UPDATE, DELETE ON modulo_c.CuentaPorPagar
```

```
FROM 'rol_tesorero';
```

```
REVOKE INSERT, UPDATE, DELETE ON modulo_c.PagoProveedor
```

```
FROM 'rol_tesorero';
```

```
REVOKE INSERT, UPDATE, DELETE ON modulo_c.FacturaProveedor
```

```
FROM 'rol_tesorero';
```

Permisos para Administrador del sistema

El Administrador del sistema tiene acceso completo a los procedimientos del módulo. Además, puede consultar la bitácora de auditoría, el log de sincronización y ejecutar funciones administrativas.

```
GRANT EXECUTE ON modulo_c.*
```

```
TO 'rol_admin_modulo_c';
```

```
GRANT SELECT ON modulo_c.*
```

```
TO 'rol_admin_modulo_c';
```

```
GRANT INSERT, UPDATE, DELETE ON modulo_c.*
```

```
TO 'rol_admin_modulo_c';
```

El Administrador también puede revisar auditoría y errores de integración:

```
GRANT SELECT ON modulo_c.BitacoraAuditoria
```

```
TO 'rol_admin_modulo_c';
```

```
GRANT SELECT ON modulo_c.LogSincronizacion
```

```
TO 'rol_admin_modulo_c';
```

En el caso de facturas excepcionales sin orden de compra, el Administrador es el rol autorizado para aprobar o rechazar la excepción, de acuerdo con el CU-17 del documento.

Asignación de roles a usuarios

Los siguientes usuarios son ejemplos de asignación. Los nombres pueden adaptarse al entorno real.

```
CREATE USER IF NOT EXISTS 'jefe_compras_c'@'%'
```

```
IDENTIFIED BY 'JefeCompras_C_2026!';
```

```
CREATE USER IF NOT EXISTS 'tesorero_c'@'%'
```

```
IDENTIFIED BY 'Tesorero_C_2026!';
```

```
CREATE USER IF NOT EXISTS 'admin_modulo_c'@'%'
```

```
IDENTIFIED BY 'Admin_C_2026!';
```

```
GRANT 'rol_jefe_compras' TO 'jefe_compras_c'@'%';
```

```
GRANT 'rol_tesorero' TO 'tesorero_c'@'%';
```

```
GRANT 'rol_admin_modulo_c' TO 'admin_modulo_c'@'%';
```

```
SET DEFAULT ROLE 'rol_jefe_compras' TO 'jefe_compras_c'@'%';
```

```
SET DEFAULT ROLE 'rol_tesorero' TO 'tesorero_c'@'%';
```

```
SET DEFAULT ROLE 'rol_admin_modulo_c' TO 'admin_modulo_c'@'%';
```

7.2 Estrategia de respaldo y restauración

La estrategia de respaldo busca proteger la estructura, los datos, procedimientos almacenados, funciones, vistas y registros de auditoría del Módulo C.

Se recomienda realizar respaldos:

- antes de ejecutar cambios estructurales;
- antes de cargar datos masivos;
- antes de pruebas de integración con otros módulos;
- al finalizar una jornada de trabajo o fase importante del proyecto.

Respaldo completo del módulo

```
mysqldump -u root -p \  
  
--databases modulo_c \  
  
--routines \  
  
--triggers \  
  
--events \  
  
--single-transaction \  
  
--default-character-set=utf8mb4 \  
  
> backup_modulo_c_completo.sql
```

Este respaldo incluye:

- estructura de tablas;
- datos;
- vistas;
- procedimientos almacenados;
- funciones;
- eventos;
- triggers, si existieran.

Aunque el diseño final del módulo evita depender de triggers para la lógica principal, se incluye --triggers para respaldar completamente cualquier elemento existente del esquema.

Respaldo solo de estructura

```
mysqldump -u root -p \  
  
--databases modulo_c \  
  
--no-data \  
  
--routines \  

```

```
--triggers \  
--events \  
--default-character-set=utf8mb4 \  
> backup_modulo_c_estructura.sql
```

Este respaldo sirve para conservar únicamente la definición del módulo sin los datos.

Respaldo solo de datos

```
mysqldump -u root -p \  
modulo_c \  
--no-create-info \  
--single-transaction \  
--default-character-set=utf8mb4 \  
> backup_modulo_c_datos.sql
```

Este respaldo sirve cuando se desea conservar únicamente la información almacenada en las tablas.

Restauración del respaldo completo

```
mysql -u root -p < backup_modulo_c_completo.sql
```

Restauración solo de datos

```
mysql -u root -p modulo_c < backup_modulo_c_datos.sql
```

7.3 Consideraciones adicionales de seguridad

Auditoría de operaciones sensibles

El módulo utiliza la tabla BitacoraAuditoria para registrar operaciones importantes sobre datos financieros. Esta medida cumple con RNF04, donde se indica que toda operación que modifique datos financieros debe guardar usuario, fecha, hora y valores anteriores o posteriores.

Las operaciones que deben quedar auditadas son:

- ingreso de facturas de proveedor;
- generación de cuentas por pagar;
- registro de pagos;
- actualización de saldos;
- programación de pagos;
- actualización de retenciones;
- actualización de cuentas vencidas;
- autorización de facturas excepcionales sin orden de compra.

Consulta de revisión de auditoría:

SELECT

tabla_afectada,

operacion,

id_registro,

usuario,

valor_anterior,

valor_nuevo,

fecha_evento

FROM modulo_c.BitacoraAuditoria

ORDER BY fecha_evento DESC;

Prevención de inyección SQL

Las operaciones críticas del módulo se ejecutan mediante procedimientos almacenados parametrizados. Esto reduce el riesgo de inyección SQL, porque los valores ingresados por el usuario se reciben como parámetros y no se concatenan como texto dentro de sentencias SQL dinámicas.

Ejemplos de procedimientos protegidos por parámetros:

CALL modulo_c.sp_ingresar_factura_proveedor(...);

CALL modulo_c.sp_registrar_pago(...);

CALL modulo_c.sp_programar_pago(...);

CALL modulo_c.sp_crear_orden_compra(...);

Además, se restringe el acceso directo a tablas principales mediante REVOKE, obligando a que los usuarios operen a través de procedimientos donde existen validaciones, transacciones y auditoría.

Enmascaramiento de datos sensibles de proveedores

La tabla Proveedor puede contener información sensible como identificación, RUC/cédula y cuenta bancaria. Para consultas generales se puede crear una vista segura que muestre estos datos parcialmente enmascarados.

```
CREATE OR REPLACE VIEW modulo_c.vw_proveedores_seguro AS
```

```
SELECT
```

```
    id_proveedor,
```

```
    tipo_proveedor,
```

```
    CONCAT(LEFT(identificacion, 3), '*****', RIGHT(identificacion, 3))
```

```
        AS identificacion_enmascarada,
```

```
    razon_social,
```

```
    email,
```

```
    telefono,
```

```
    direccion,
```

```
    dias_credito,
```

```
    CONCAT('****', RIGHT(cuenta_bancaria, 4))
```

```
        AS cuenta_bancaria_enmascarada,
```

```
    banco,
```

```
    calificacion,
```

```
    activo
```

```
FROM modulo_c.Proveedor
```

```
WHERE activo = 1;
```

Permisos sobre la vista segura:

```
GRANT SELECT ON modulo_c.vw_proveedores_seguro
```

```
TO 'rol_jefe_compras';
```


GRANT SELECT ON modulo_c.vw_proveedores_seguro

TO 'rol_tesorero';

Con esta vista, los usuarios pueden consultar información general de proveedores sin visualizar completamente datos sensibles.

Sección 8: Integración con otros módulos

Descripción de los puntos de integración del módulo con los demás equipos. Entidades compartidas y cómo se mantiene la integridad referencial entre módulos. Prueba de al menos una consulta o proceso que involucre datos de otro módulo.

8.1 Descripción general de la integración

El Módulo C — Cuentas por Pagar se integra con los demás módulos del SIGE mediante referencias lógicas, vistas de integración, procedimientos almacenados y registros de sincronización. La finalidad de esta integración es permitir que el ciclo de compras se conecte con inventarios, facturación y gestión tributaria sin romper la independencia funcional de cada módulo.

El flujo principal del Módulo C es:

Orden de compra → Recepción de mercadería → Factura de proveedor → Cuenta por pagar → Pago

Dentro de ese flujo, el Módulo C se comunica principalmente con:

- **Módulo B — Inventarios**, para consultar productos y notificar entradas de mercadería.
- **Módulo D — Impuestos**, para enviar datos de facturas de proveedor y recibir comprobantes de retención.
- **Módulo A — Facturación y Cuentas por Cobrar**, de forma indirecta, exponiendo costos de adquisición para que el módulo de facturación pueda usarlos como referencia en análisis de precios o márgenes.

En los scripts actuales, el Módulo B define la tabla productos como entidad maestra compartida para los módulos A y C, con la clave primaria producto_id. También maneja movimientos_inventario, donde se registran entradas, salidas y ajustes de inventario.

Por su parte, el Módulo C mantiene referencias al producto mediante campos como ref_mod_b_id_producto en DetalleOrdenCompra y DetalleRecepcion, además de vistas y procedimientos de integración como vw_recepciones_pendientes_notificar, sp_marcar_recepcion_notificada, sp_consultar_precio_costo_producto y sp_actualizar_retencion_desde_modd.

8.2 Puntos de integración del Módulo C

8.2.1 Integración con Módulo B — Inventarios

La integración con el Módulo B ocurre en dos momentos principales.

Primero, cuando el Módulo C crea una orden de compra, los productos solicitados se registran en `DetalleOrdenCompra` usando el campo `ref_mod_b_id_producto`. Este campo representa el identificador del producto definido por el Módulo B. Además, el Módulo C guarda una copia del código, descripción, unidad de medida y precio acordado al momento de crear la orden. Esto permite conservar el historial de la compra aunque posteriormente cambien los datos del producto en el inventario.

Segundo, cuando se registra una recepción de mercadería, el Módulo C genera registros en `Recepcion` y `DetalleRecepcion`. Luego, el procedimiento `sp_registrar_recepcion` deja pendiente una notificación hacia el Módulo B mediante `LogSincronizacion`, usando la interfaz IF-03. Esta notificación indica que el inventario debe actualizarse con una entrada de mercadería.

En los scripts actuales del Módulo B, la actualización de inventario se representa mediante la tabla `movimientos_inventario`, que registra el producto, bodega, tipo de movimiento, cantidad y referencia. El campo `referencia` está pensado para guardar datos como el número de factura o el identificador de recepción proveniente del Módulo C.

8.2.2 Integración con Módulo D — Impuestos

La integración con el Módulo D se da durante el ingreso de facturas de proveedor.

Cuando el Módulo C registra una factura, esta queda inicialmente en estado `pendiente_retencion`. Luego, la vista `vw_facturas_pendientes_retencion` expone los datos necesarios para que el Módulo D pueda calcular retenciones y registrar el IVA pagado. Esta vista incluye información como identificador de factura, RUC del proveedor, base imponible, IVA, tipo de bien o servicio, código de sustento y fecha.

El Módulo D, en su script actual, contiene tablas como `tarifas_iva`, `retenciones_iva_config`, `retenciones_renta_config` y `comprobantes_retencion`. La tabla `comprobantes_retencion` incluye referencias lógicas hacia el Módulo C mediante los campos `ref_mod_c_proveedor_id` y `ref_mod_c_factura_id`. Esto permite relacionar el comprobante tributario generado por el Módulo D con la factura de proveedor registrada en el Módulo C.

Cuando el Módulo D devuelve el XML de retención y el número de autorización, el Módulo C actualiza la factura mediante el procedimiento `sp_actualizar_retencion_desde_modd`. Este procedimiento registra el XML, el número de autorización y cambia el estado de la factura a `retencion_generada`.

8.2.3 Integración con Módulo A — Facturación y Cuentas por Cobrar

La integración con el Módulo A es indirecta. El Módulo A utiliza productos para emitir facturas de venta, mientras que el Módulo C registra los costos de adquisición de esos productos mediante órdenes de compra.

El Módulo C expone el costo de adquisición mediante los procedimientos:

CALL modulo_c.sp_consultar_precio_costo_producto(id_producto);

CALL modulo_c.sp_consultar_ultimo_costo_adquisicion(id_producto, limite);

Estos procedimientos permiten que el Módulo A consulte el costo de compra más reciente de un producto sin acceder directamente a las tablas internas del Módulo C. Esto mantiene bajo acoplamiento entre módulos y permite que el Módulo A use la información para análisis de margen, precio de venta o reportes internos.

En el script del Módulo A, la tabla detalle_factura incluye un campo producto_id, relacionado conceptualmente con el producto del Módulo B. Por lo tanto, tanto el Módulo A como el Módulo C se apoyan en la entidad productos administrada por el Módulo B.

8.3 Entidades compartidas e integridad referencial entre módulos

Las principales entidades compartidas o relacionadas entre módulos son las siguientes:

Entidad / referencia	Módulo propietario	Uso en Módulo C	Forma de integración
productos	Módulo B	Se referencia en órdenes y recepciones mediante ref_mod_b_id_producto.	Referencia lógica al catálogo de inventario.
movimientos_inventario	Módulo B	Recibe entradas cuando el Módulo C registra una recepción.	Proceso de integración IF-03.
tarifas_iva	Módulo D	Se referencia mediante ref_mod_d_tarifa_id	Referencia lógica tributaria.

		en facturas de proveedor.	
comprobantes_retencion	Módulo D	Se relaciona con facturas y proveedores del Módulo C mediante referencias lógicas.	Integración IF-05 / retenciones.
FacturaProveedor	Módulo C	Envía datos al Módulo D para retención.	Vista vw_facturas_pendientes_retencion.
DetalleOrdenCompra	Módulo C	Guarda producto solicitado, cantidad y precio pactado.	Usa ref_mod_b_id_producto como referencia al producto.
DetalleRecepcion	Módulo C	Guarda cantidad recibida por producto.	Sirve como base para actualizar inventario.
LogSincronizacion	Módulo C	Registra integraciones pendientes, exitosas o fallidas.	Control de comunicación entre módulos.

La integridad referencial interna del Módulo C se mantiene mediante claves foráneas entre sus propias tablas: Proveedor, OrdenCompra, DetalleOrdenCompra, Recepcion, DetalleRecepcion, FacturaProveedor, CuentaPorPagar y PagoProveedor.

Para las relaciones entre módulos, la estrategia usada es una **integridad referencial lógica**. Esto significa que el Módulo C no depende directamente de modificar tablas internas de otros módulos, sino que conserva identificadores de referencia y usa interfaces o vistas de integración. Esta decisión reduce el acoplamiento y evita que un cambio interno en un módulo rompa directamente la lógica del otro.

En el caso del Módulo B, el Módulo C no modifica directamente el catálogo de productos. Solo almacena el identificador del producto y genera una notificación pendiente para que la capa de integración actualice el inventario. En el caso del Módulo D, el Módulo C expone las facturas pendientes de retención y luego recibe la respuesta tributaria mediante un procedimiento específico.

8.4 Prueba de integración con otro módulo

Proceso probado: recepción de mercadería del Módulo C y actualización de inventario en Módulo B

Esta prueba representa la integración entre el Módulo C y el Módulo B. El objetivo es demostrar que una recepción registrada en C puede ser usada para generar un movimiento de entrada en el inventario del Módulo B.

El Módulo C expone las recepciones pendientes de notificar mediante la vista:

modulo_c.vw_recepciones_pendientes_notificar

Esta vista contiene el identificador de la recepción, el identificador de la orden de compra, el producto recibido y la cantidad recibida. Esa información se cruza con modulo_b.productos para verificar que el producto recibido existe en el inventario.

Consulta de verificación previa

```
SELECT
    r.id_recepcion,
    r.id_oc,
    r.ref_mod_b_id_producto AS producto_id,
    p.product_id AS id_producto_modulo_b,
    p.codigo,
    p.descripcion,
    r.cantidad_recibida,
    p.stock_actual,
    p.stock_min
FROM modulo_c.vw_recepciones_pendientes_notificar r
JOIN modulo_b.productos p
    ON p.product_id = r.ref_mod_b_id_producto
WHERE p.activo = 1
ORDER BY r.id_recepcion
```

LIMIT 0, 1000;

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	id_recepcion	id_oc	producto_id	id_producto_modulo_b	codigo	descripcion	cantidad_recibida	stock_actual	stock_min

Interpretación:

Esta consulta valida que las recepciones pendientes del Módulo C correspondan a productos existentes y activos en el Módulo B. Si la consulta devuelve registros, significa que existen recepciones listas para ser procesadas como entradas de inventario.

Proceso de actualización de inventario

-- PROCESO DE ACTUALIZACIÓN DE INVENTARIO

-- Integración Módulo C -> Módulo B

-- Versión corregida usando product_id

-- Verificar primero qué recepción se va a procesar

SELECT

@id_recepcion AS id_recepcion_a_procesar;

-- Verificar que la recepción esté pendiente de notificar

SELECT

r.id_recepcion,

r.id_oc,

r.ref_mod_b_id_producto AS product_id,

r.cantidad_recibida

FROM modulo_c.vw_recepciones_pendientes_notificar r

WHERE r.id_recepcion = @id_recepcion;

START TRANSACTION;

-- 1. Registrar movimiento de entrada en el Módulo B

```
INSERT INTO modulo_b.movimientos_inventario (
    product_id,
    bodega_id,
    tipo,
    cantidad,
    fecha,
    referencia,
    observaciones
)
SELECT
    r.ref_mod_b_id_producto AS product_id,
    1 AS bodega_id,
    'ENTRADA' AS tipo,
    r.cantidad_recibida,
    NOW() AS fecha,
    CONCAT('REC-C-', r.id_recepcion) AS referencia,
    CONCAT('Entrada generada desde recepción del Módulo C, OC ', r.id_oc) AS
observaciones
FROM modulo_c.vw_recepciones_pendientes_notificar r
JOIN modulo_b.productos p
    ON p.product_id = r.ref_mod_b_id_producto
WHERE r.id_recepcion = @id_recepcion
    AND p.activo = 1;
```

-- 2. Actualizar stock actual del producto en Módulo B

```

UPDATE modulo_b.productos p
JOIN (
    SELECT
        ref_mod_b_id_producto AS product_id,
        SUM(cantidad_recibida) AS cantidad_total
    FROM modulo_c.vw_recepciones_pendientes_notificar
    WHERE id_recepcion = @id_recepcion
    GROUP BY ref_mod_b_id_producto
) x
ON x.product_id = p.product_id
SET p.stock_actual = p.stock_actual + x.cantidad_total;

```

-- 3. Marcar la recepción como notificada en Módulo C

```

CALL modulo_c.sp_marcar_recepcion_notificada(
    @id_recepcion,
    'integracion_modb'
);

```

```

COMMIT;

```

Consulta de verificación posterior

```

-- CONSULTA DE VERIFICACIÓN POSTERIOR
-- Verifica movimiento generado y stock actualizado

```

```

SELECT
    p.product_id,
    p.codigo,

```



```
p.descripcion,  
p.stock_actual,  
mi.movimiento_id,  
mi.tipo,  
mi.cantidad,  
mi.referencia,  
mi.fecha
```

```
FROM modulo_b.productos p
```

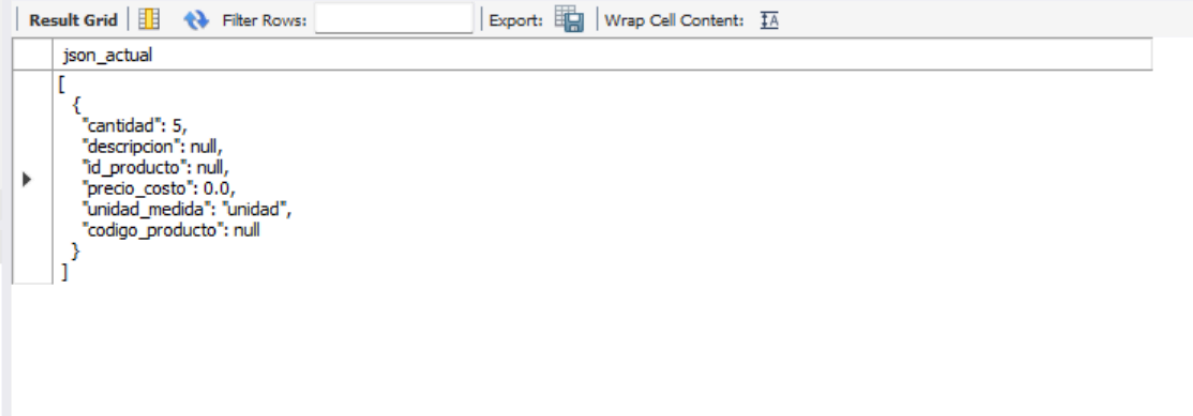
```
JOIN modulo_b.movimientos_inventario mi
```

```
ON mi.product_id = p.product_id
```

```
WHERE mi.referencia = CONCAT('REC-C-', @id_recepcion);
```

Resultado esperado:

La consulta debe mostrar un movimiento de inventario de tipo ENTRADA, asociado al producto recibido desde el Módulo C. Además, el stock actual del producto debe haberse incrementado según la cantidad recibida.



The screenshot shows a database query result grid. The top bar includes 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'. The main area displays a JSON array under the column header 'json_actual'. The array contains one object with the following fields: 'cantidad' (5), 'descripcion' (null), 'id_producto' (null), 'precio_costo' (0.0), 'unidad_medida' ('unidad'), and 'codigo_producto' (null).

json_actual
[{ "cantidad": 5, "descripcion": null, "id_producto": null, "precio_costo": 0.0, "unidad_medida": "unidad", "codigo_producto": null }]

```
'[\n {\n  \"cantidad\": 5,\n  \"descripcion\": null,\n  \"id_producto\": null,\n  \"precio_costo\": 0.0,\n  \"unidad_medida\": \"unidad\",\n  \"codigo_producto\": null\n }\n]'
```

Verificación en el Módulo C

```
SELECT
```

```

id_recepcion,

id_oc,

notificado_inventario,

fecha_notificacion_inventario,

usuario_notificacion

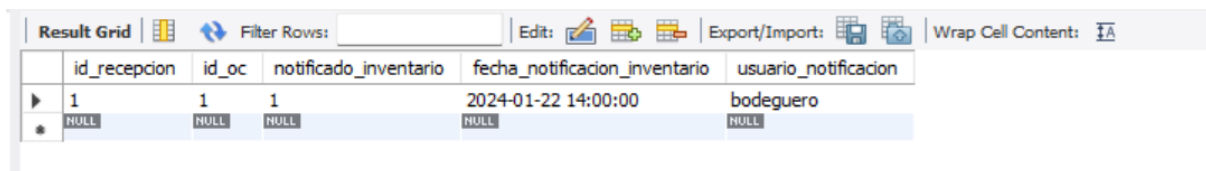
FROM modulo_c.Recepcion

WHERE id_recepcion = 1;

```

Resultado esperado:

El campo notificado_inventario debe quedar en 1, indicando que la recepción ya fue procesada por la integración con el Módulo B. También debe registrarse la fecha de notificación y el usuario de integración.



	id_recepcion	id_oc	notificado_inventario	fecha_notificacion_inventario	usuario_notificacion
▶	1	1	1	2024-01-22 14:00:00	bodeguero
*	NULL	NULL	NULL	NULL	NULL

Nota: En la base de datos ejecutada, el identificador de producto del Módulo B se encuentra como product_id; por ello, la consulta de integración usa ese nombre físico de columna, aunque conceptualmente corresponde al producto compartido entre Módulo B y Módulo C.

8.5 Consideración técnica sobre compatibilidad de nombres

Al revisar los scripts actuales de los otros módulos, se observa que algunas consultas de integración antiguas usan nombres de columnas que no coinciden completamente con la versión actual del Módulo C y del Módulo B.

Por ejemplo, el Módulo B actual define la clave primaria de productos como:

producto_id

mientras que algunas referencias antiguas mencionan product_id. De igual forma, en el Módulo C actual la vista detalle_orden_compra proviene de DetalleOrdenCompra, cuya referencia al producto se llama:

ref_mod_b_id_producto

y no producto_id.

Por esa razón, para la integración final se debe usar la siguiente correspondencia:

Referencia antigua	Referencia correcta en scripts actuales
product_id	producto_id
doc.producto_id	doc.ref_mod_b_id_producto
oc.orden_id	oc.id_oc
doc.orden_id	doc.id_oc

Esta aclaración evita errores al ejecutar consultas cruzadas entre los módulos y mantiene coherencia con el script actual del Módulo C y la versión actual del Módulo B.

Conclusión de la integración

La integración del Módulo C con los demás módulos se basa en interfaces controladas y referencias lógicas. Con el Módulo B, el Módulo C consulta productos y notifica recepciones para actualizar inventario. Con el Módulo D, expone facturas pendientes de retención y recibe la información tributaria generada. Con el Módulo A, proporciona información de costos de adquisición para apoyar consultas de precios o márgenes.

La integridad se mantiene mediante claves foráneas internas dentro del Módulo C, uso de identificadores compartidos, vistas de integración, procedimientos almacenados y la tabla LogSincronizacion, que permite controlar eventos pendientes, exitosos o fallidos entre módulos. La prueba presentada demuestra que una recepción registrada en el Módulo C puede generar una entrada de inventario en el Módulo B y cerrar correctamente el ciclo marcando la recepción como notificada.

Sección 9: Repositorio

URL del repositorio (GitHub/GitLab) con: historial de commits que muestre contribución de cada integrante, README con instrucciones de instalación y carga del script, estructura de carpetas ordenada (diagramas, scripts DDL, DML, objetos, consultas, informe).

<https://github.com/arilofo/M-dulo-C>

Sección 10: Conclusiones y reflexión técnica

Síntesis de aprendizajes por integrante. Decisiones técnicas más relevantes y su justificación. Dificultades encontradas y cómo se resolvieron. Propuestas de mejora si el sistema fuera a producción.

- **Daniel**

En lo personal, este proyecto me permitió profundizar mucho más en el diseño y modelado de bases de datos, especialmente en aspectos que no se abordaron con tanto detalle durante las clases. Si bien en la materia se trabajaron conceptos fundamentales como normalización, relaciones y consultas SQL, el desarrollo del Módulo C me llevó a comprender de manera más práctica temas como la especialización en modelos EER, la integridad referencial entre módulos, el manejo de transacciones y la optimización de consultas mediante índices y análisis de ejecución.

En cuanto a las dificultades encontradas, considero que una de las más importantes fue la documentación técnica del proyecto. Explicar de manera clara el funcionamiento del modelo EER, justificar las decisiones de diseño, describir restricciones de integridad y documentar procedimientos SQL resultó más complejo de lo esperado. A diferencia de escribir código o diseñar tablas, redactar documentación técnica comprensible y bien estructurada requiere mayor precisión y organización, algo que este proyecto me ayudó a desarrollar significativamente.

Finalmente, considero que este trabajo me permitió comprender con mayor profundidad la importancia del diseño integral de una base de datos dentro de un sistema empresarial real. Aspectos como la separación entre facturas, cuentas por pagar y pagos parciales, así como la integración entre módulos mediante claves foráneas e interfaces lógicas, ayudaron a entender cómo un sistema puede mantenerse organizado, escalable y consistente. Además, el proyecto permitió reforzar conocimientos sobre normalización, control de concurrencia y diseño relacional aplicado a escenarios empresariales más cercanos a un entorno profesional.

- **Marco**

Durante el desarrollo del módulo C, el mayor reto técnico fue garantizar la integridad referencial en un sistema que depende constantemente de módulos externos como Ventas y Facturación. Esto obligó a diseñar cada tabla y transacción no de forma aislada, sino como parte de un ecosistema donde una decisión mal tomada podía romper la consistencia de todo el flujo financiero. Para mitigarlo, se establecieron restricciones claras a nivel de base de datos y se validó continuamente el comportamiento del modelo frente a operaciones cruzadas entre módulos.

En el plano del rendimiento, trabajar con MySQL profundizó mi comprensión sobre el análisis de consultas mediante EXPLAIN, convirtiendo lo que parecía un resultado más del motor en un mapa real para detectar escaneos innecesarios e índices ausentes. En paralelo, la implementación de roles de usuario permitió delimitar con precisión qué operaciones puede ejecutar cada actor sobre los datos financieros, y la adopción de backups periódicos dejó de ser un concepto teórico para volverse una práctica concreta entendiendo el impacto real que tiene la pérdida de datos en un módulo de cobros.

Como reflexión final, la decisión de no usar GitHub en esta etapa fue pragmática y nos permitió avanzar sin fricciones mientras el equipo consolidaba su dominio sobre la base de datos, aunque deja como lección clara que el control de versiones debe adoptarse desde el inicio en proyectos futuros. El desarrollo de este módulo demostró que construir un sistema financiero robusto no solo exige dominio técnico, sino también criterio para tomar decisiones de diseño conscientes y anticipar los puntos de falla antes de que ocurran.

- **Ariel**

Durante la realización del proyecto he podido aprender cómo es trabajar en diferentes módulos, de la manera en la que se reparte el trabajo y el cómo se necesita la comunicación continua entre los diferentes módulos, he visto que el uso de los contratos es algo realmente importante al momento de coordinar avances, pues con esto se fijan fechas límite para poder avanzar de manera uniforme entre todos los módulos y que ninguno quede atrasado de manera significativa lo que es bastante útil al momento de necesitar avances específicos de otros módulos.

Las dificultades que se presentaron fueron parecidas a las del proyecto en ingeniería de software, por lo que hubieron problemas al momento de que no hubo suficiente guía cuando se realizaba el proyecto lo que llevaba más tiempo, pues debíamos investigar para poder resolver estas dudas, también llegó a ser un problema la necesidad de comunicación del proyecto, pues algunas veces las respuestas de los módulos se hacían esperar y al momento que necesitaban cierto avances de los otros módulos, perdíamos el tiempo al estar esperando la respuesta.

Como recomendaciones considero que la implementación de un canal de comunicación en el que se encuentre presente el docente, de esta manera se evidencia las fechas en las que se solicita información y las fechas en las que se recibe la respuesta; además considero implementar más tareas similares a los entregables y los contratos, estos fueron de gran ayuda durante la realización del proyecto, pues presionaba a los módulos a tener que avanzar hasta cierto punto y esto aseguraba que los módulos no se atrasen.